

Engineering Efficient Self-Play Chess: Search, Replay, and Throughput Under Limited Compute

Bertil Braun

contact@bertil-braun.de

Abstract

How strong can an AlphaZero-style chess system become under limited training compute when its entire learning loop is engineered for efficiency? We study a model trained from random initialization through searched self-play on one eight-GPU node. Its accepted training lineage spans 60 effective hours and ingests 3.25 million completed games, about 209 million net replay positions, and 836.6 million training presentations. The selected 6.3-million-parameter model reaches 3,114 benchmark Elo [3,065, 3,163] at 10,000 searches and 3,251 [3,206, 3,297] at 100,000 searches on a fixed-node Stockfish 13 ladder. An estimator-matched 64-search comparison places its late training plateau approximately 74 Elo above the previous baseline. We examine search allocation, replay and restart-state selection, policy representation, progressive sizing, quantized inference, and throughput engineering, including approaches that failed or did not justify their cost. The resulting strength belongs to the integrated system; component proxies are not treated as isolated causal Elo gains. All ratings are protocol-specific benchmark estimates, not FIDE or unrestricted-engine ratings.

Keywords: AlphaZero, chess, self-play, Monte Carlo tree search, replay, inference

1. How far can efficient self-play go?

An AlphaZero-style chess player improves by searching its own games, learning from the resulting positions, and repeating that cycle with a stronger network. Each step consumes compute. Search creates targets, training absorbs them, and evaluation must distinguish real progress from noise. When all three share one eight-GPU node, a faster model forward or a cheaper search matters only if it produces stronger play sooner.

This study asks how strong that loop can become under a very limited compute budget when the entire system is engineered for efficiency. The selected model was trained from random initialization using self-play, without human games or pretrained chess weights. It has 6.3 million parameters. Under the project's fixed-node Stockfish 13 calibration it measured **1,658 benchmark Elo without search** and **3,251 benchmark Elo at 100,000 searches per move**. The previous four-day training baseline trails the final recipe by approximately **74 Elo** in an estimator-matched 64-search plateau comparison. These numbers describe the stated match protocols; they are not FIDE ratings or unrestricted-engine rankings. The full intervals and artifacts appear in [the results chapter](#).

The most useful finding is the interaction among decisions. Search determines which targets are worth paying for. Replay determines which of those targets the learner sees again. The network representation determines both what can be learned and how quickly positions can be evaluated. Batching, native search, and TensorRT make enough games possible for those choices to matter at all. A change that improved one local proxy sometimes worsened the complete learning loop.

Scope and contributions

Chess is the research subject. The same runtime supports Go on 7×7 and 9×9 boards, and a small-board baseline [validated the shared platform](#). Go also supplied ideas, notably KataGo's fast and full searches. The project briefly considered small-board Go as a cheaper place to tune parameters for chess. The basic loop worked, but its large

first-player advantage, shorter games, rapidly learned value target, and apparent need for different tuning made that transfer unattractive. This is the project owner's qualitative rationale, not a controlled cross-game result. Go receives no separate strength claim in this report.

The report contributes:

1. A complete, reproducible chess self-play run whose readable recipe starts at [chess-final-config.yaml](#) and whose selected checkpoint, evaluation, and hashes are frozen in the [final result record](#).
2. Substantial investigations of search allocation, graph search, inference caching, policy representation, model sizing, replay, restart states, resignation, auxiliary targets, and quantized serving. Each conclusion is bounded by the workload and evidence actually measured.
3. An account of the throughput path that made the experiment feasible: native C++ game and tree ownership, batched GPU inference, TensorRT deployment, columnar replay, and persistent distributed training.
4. Three failures with transferable lessons about self-play targets, deployment correctness, and model promotion.

The final recipe combines many changes. Its strength establishes the outcome of the assembled system, not an isolated Elo contribution for every retained component. The report uses paired games for playing-strength claims, keeps throughput and target-fidelity measurements attached to their own protocols, and labels owner recollections when original result artifacts are unavailable.

Reading the report

[Chapter 2](#) gives the compact measurement rules. [Chapter 3](#) shows the learning loop. The investigation chapters then follow the three central choices: how to spend search, what the network predicts, and which positions become training data. A short systems chapter explains how those choices were made affordable. The three failure studies lead into the integrated recipe and the final measured outcome.

Detailed implementation history remains in the linked repository evidence rather than the paper's narrative.

2. How claims are measured

The unit of evidence depends on the claim. Faster inference can support a throughput conclusion. Lower policy loss can show that a target is learnable. Neither by itself shows that the player became stronger. This distinction is especially important in a self-play system: a saved simulation can be absorbed by concurrent training, and a network that fits stored targets can still create a worse game distribution.

Playing strength

The project evaluates chess models against Stockfish 13 at fixed node limits. Openings are paired: each starting position is played once from each colour. Every reported match identifies the candidate checkpoint and deployed inference artifact, candidate search budget and parallelism, opponent limit, opening suite, W/D/L count, score, and uncertainty interval. The terminal matrix and its raw evidence are in the [final result record](#).

Absolute ratings use [Marco Meloni's fixed-node Stockfish calibration](#). They are **benchmark Elo** under this match protocol. They are not FIDE ratings, online ratings, or ratings against unrestricted contemporary engines. For each model search budget, the headline uses the opponent limit whose match score is closest to 0.5, reducing Elo extrapolation. Both tested limits remain visible. Their inferred ratings disagree most at the shallowest model budget; the data do not identify a single cause for that disagreement.

The policy-only row uses direct masked-policy action selection through a float TorchScript export. Searched rows use the INT8 TensorRT deployment artifact. Those rows answer different questions and retain their artifact identities. The deep-search curve also changes search parallelism, so it is an operating curve rather than a controlled fixed-parallelism scaling law.

Learning and comparison

An online learning comparison should measure playing strength over time while keeping initial model weights, replay contents, evaluation, and hardware workload as close as possible. When an intervention is small, independent from-scratch trajectories can differ more than the proposed effect. The decisive adaptive-search comparisons therefore forked the same model and replay state. A frozen-replay fit is useful for screening an optimizer, policy head, or quantization scheme, but it cannot establish self-play Elo by itself.

The cross-campaign 64-search plot shows descriptive training trajectories. The previous baseline and final recipe used different ladder estimators, so the report does not

subtract their plotted endpoints as a strength claim. A retrospective comparison on a matched three-rung estimator estimates an approximately 74-Elo final plateau gain, with a separate transfer sensitivity of about ± 15 Elo. The latter is not a game-level confidence interval. The [derivation and trimmed plot input](#) record the comparison boundary.

Throughput and target fidelity

The report names each throughput stage: model evaluations, search simulations, completed games, admitted replay positions, optimizer steps, and Elo per wall-clock hour. A gain at one stage may shrink or disappear at the next. Batch size, concurrent games, GPU contention, search parallelism, and training overlap therefore belong with any rate claim. Saturated many-game service time must not be called interactive single-game latency.

Serving artifacts are checked as chess models, not merely as valid files. The TensorRT failure investigation showed that successful export and refit calls could still produce incorrect legal move probabilities. Fidelity checks use real encoded positions, legal-action masking, policy agreement and KL, and value error. A float checkpoint's match cannot stand in for a different INT8 artifact when INT8 is the intended deployment.

Provenance and limits

The living [final configuration](#) explains the current recipe. The reported run additionally needs its frozen source revision, resolved configuration, checkpoint and deployment hashes, evaluation assets, and archived results. The [evidence index](#) links those identities. A plan records intent; only implemented code and preserved measurements support claims that an experiment happened.

Many choices in the successful recipe were changed together. The report calls them *retained* when they are in that recipe, and gives an isolated effect only when a controlled comparison supports one. Missing historical policy-head artifacts and qualitative owner recollections are stated as such. The [experiment ledger](#) and [coverage matrix](#) retain the full audit trail without making the reader traverse it to understand the main result.

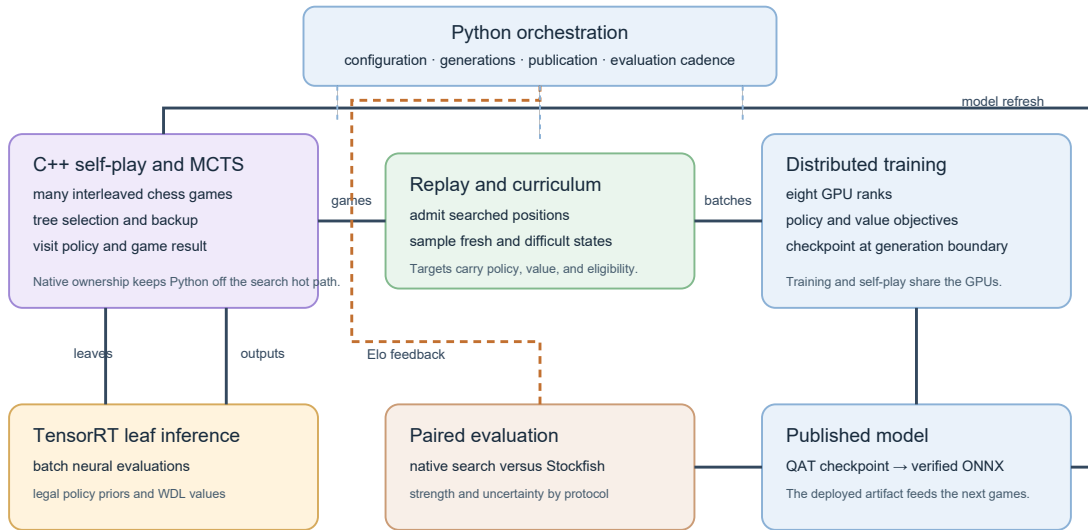
3. System and training method

Runtime boundary

The systems design matters here for one reason: AlphaZero training needs enough searched games and fresh positions to learn within the available wall-clock budget. The report therefore describes the throughput-critical path—native search, batching, inference, replay, and training ownership—while leaving topology sweeps and recovery machinery to the linked engineering record.

One learning loop, one shared compute budget

Throughput matters when it delivers more useful positions and measurable strength per hour.



Solid arrows: data or model flow. Dashed orange arrow: evaluation feedback to selection.

Diagram shows the normal training loop; source, checkpoint, and match identities are recorded separately.

Figure 3.1. A published model returns to batched native self-play; searched positions become replay targets for the trainer. Paired evaluation measures the published checkpoint and feeds selection decisions back to Python. The diagram shows ownership and data flow, not the number or scheduling of every worker.

The system has one native implementation of the latency-sensitive path and one typed orchestration layer:

- `cpp/` owns chess and Go state, legal actions, encodings, batched inference, MCTS, self-play, and interactive analysis;
- `py/` owns validated experiment configuration, worker lifecycle, replay ingestion, distributed training, checkpoint publication, evaluation, telemetry, and tools.

This division avoids maintaining a second production search or rules implementation in Python. Cross-language contracts—action mapping, tensor shape, feature planes, symmetry, and output order—are tested explicitly. The [Python runtime architecture](#) is the detailed ownership record.

Chess representation and outputs

The network receives a board-aligned tensor containing current state, history, and additional chess features. The action space contains 1,880 reduced chess actions. Game-defined symmetry transforms the state and every action-space target together. The production inference artifact exports only the primary policy and WDL/value outputs; auxiliary training heads are stripped from serving artifacts.

The final architecture family is convolutional. Residual blocks use scaled post-activation with capped activations, and global-pooling context is inserted every second block. A from-to attention policy head scores move origin and destination structure far more compactly than the earlier dense spatial projection. The value path uses two value channels and a small fully connected layer. Exact stage shapes are listed in [the final recipe](#).

Search and self-play

Self-play uses PUCT-style Monte Carlo tree search with neural policy priors and WDL/value estimates. The final recipe uses fixed per-generation visit budgets rather than per-position adaptive allocation. Root exploration uses Dirichlet noise, reduced-parent-value first-play urgency, and forced playouts. A fraction of root visits is retained when advancing the tree.

Thousands of games are interleaved so many small inference requests become large GPU batches. Native workers own the trees; one inference worker per process submits batched work to TensorRT. Search, batching, and worker topology must be interpreted together: raising parallel simulations can improve device fill while changing search semantics, and excessive in-flight games can improve aggregate throughput while increasing individual-game latency.

Starting positions combine shallow random legal openings with archived restart states. Restart candidates are drawn from uncertain, consequential positions rather than uniformly from all history. This targets useful diversity while retaining a defined fraction of standard-game evolution. Calibrated resignation saves completed-game work only after its false-nonloss evidence gate is satisfied; continuation games preserve auditing and terminal training examples.

Replay and materialization

Native workers publish completed trajectories atomically. Parallel materializers reconstruct observations and write fixed-layout columnar shards. A single circular memory-mapped store is the canonical training replay. Sparse policies retain a bounded number of action entries, while dense batches are reconstructed only at the training boundary.

The layout stores the state, legal moves, primary search policy, WDL target, root value, sample weight, source generation, surprise information, and configured auxiliary targets. Append and restart behavior are designed around exactly-once claims and explicit rejection telemetry. See [Columnar replay](#) and [shard ingestion](#) and the later [materialization rework](#).

Sampling mixes uniform probability with policy surprise, emphasizing positions where search disagreed with the network prior while preserving broad coverage. Capacity grows in stages as the run matures. Credit accounting ties optimizer work to materialized samples through an explicit replay ratio instead of allowing the learner to outrun data generation silently.

Training

The final trainer uses eight persistent NCCL ranks, one per GPU, with global batch 2,048 and bfloat16 training. Training proceeds in blocking optimizer quanta while a configured subset of self-play workers remains active. A *generation* is the approachable public term for one training-and-publication cycle; a *checkpoint* is the durable model artifact written at such a boundary. This overlap turns many nominal self-play savings into slack rather than wall-clock savings, a central result of the adaptive-stopping study.

The primary objective combines policy cross-entropy and WDL/value loss. Outcome value is discounted by ply, and a small scheduled blend introduces search-root value later in training. Two training-only auxiliary targets are retained: the next position's search policy and normalized remaining game length. Their purpose is representation learning; they are not served during search.

Progressive models and publication

Training begins with a smaller network to exploit its higher early self-play throughput. In the production controller, the immediate larger candidate starts from an independent initialization and trains beside the active model. Candidate starts are triggered by an Elo-improvement plateau, and promotion requires two qualifying head-to-head matches. A separate final investigation tested function-preserving growth from the trained medium model; it is not the controller's ordinary initialization path. A promotion publishes the new model without changing replay identity or generation accounting. The full contract is in [Progressive model sizing](#).

Rank zero publishes a complete training checkpoint and a trimmed inference artifact. In the final path, QAT-aware ONNX artifacts are refit into TensorRT templates for self-play and evaluation. Numerical fidelity is a publication condition and is discussed in [Chapter 5](#).

Evaluation

Short-lived evaluation jobs run independently of training on a fixed cadence. They include fixed-dataset metrics and paired-opening Stockfish ladders for policy-only and searched play. The final evaluation is a separate terminal protocol with larger matches and deeper budgets. Engine binaries, datasets, opening books, and their hashes are configuration-owned; [evaluation engine documentation](#) records installation and identity rules.

Evaluation itself became an engineered subsystem. Adaptive ladders bracket the candidate against adjacent Stockfish node limits rather than extrapolating from one score; paired openings reverse colors; concurrent jobs are device-cycled; and reports retain raw W/D/L and exact search identity. Batch shape can alter serving behavior and reshuffle outcomes, so it is part of the protocol rather than an invisible speed setting. The [ladder-batching study](#), [strength-over-generation series](#), and [deep fixed-node match](#) show the progression from frequent noisy signal to terminal-strength measurement.

Detailed restart and recovery semantics are implementation concerns rather than a report contribution. The public reproducibility boundary is simply that completed trajectories, replay state, checkpoints, and evaluation evidence are durably committed before they are credited or reported; the linked architecture documentation owns the mechanics.

4A. Spending search where it matters

Search was the clearest way to make a fixed network play better, but also one of the most expensive parts of the learning loop. That made allocation an unusually attractive research target: if easy positions could be recognized early, their unused simulations could be moved to difficult positions or converted into more games. The project tested that premise at several levels, from simple stopping rules to learned controllers and a complete graph-search implementation. The result was not a better allocator. It was a better understanding of why the transparent fixed budget remained hard to beat.

Throughout this chapter, three outcomes are kept separate. Fixed-network playing strength asks whether search chooses stronger moves. Target fidelity asks whether a shallow policy resembles a deeper one. End-to-end learning asks whether the resulting data makes the next network improve faster. Throughput is a fourth, systems-level quantity. Success on any one of these axes did not guarantee success on the others. The experiment classifications and evidence grades used below are defined in [Chapter 2](#), with the underlying records indexed in the [search experiment ledger](#).

The fixed-budget baseline

The native engine performs policy-guided Monte Carlo tree search. The network supplies legal-move priors and a win/draw/loss value; PUCT selects leaves; neural evaluations are batched across active games; and values are backed up from the alternating player's perspective. A visit cap is deliberately unambiguous. It does not depend on a difficulty estimate, a calibrated controller, or a learned model that may become stale as training advances.

The fixed-network sweep established that this simple control was valuable. Against the same opponent, score rose from 0.318 at 200 visits to 0.537, 0.580, 0.662, and 0.748 at 400, 600, 1,000, and 1,600 visits. Across the better resolved middle and deep part of the sweep, the fitted trend was about 81 Elo per doubling. Individual adjacent comparisons were noisy, so the monotone trend is more credible than any single increment. Search was also still changing the training target: at 600 visits, only 72.4% of positions selected the same best move as the network's own 10,000-visit reference. These measurements support

spending more search when resources permit; they do not identify an optimal training schedule ([search evaluation, synthesis](#)).

The retained schedule therefore increased the cap in auditable stages—300, 400, 500, 600, and finally 800 visits—rather than presenting those boundaries as independently optimized discoveries. Even this count needs care: a retained root can begin a move with existing statistics, so a nominal visit cap is not always the amount of new work performed.

Why fast and full searches did not transfer

A KataGo-inspired scheme offered an appealing alternative. Most moves could use a cheap search to advance the game, while a random minority received a full search and became primary policy targets. In long Go games this can exchange some target density for many more independent terminal outcomes. Chess did not show the same bottleneck. Games were shorter, positions were often more decisive, and the value objective was already learning. Discarding cheap-search positions therefore reduced the supply of policy targets without a demonstrated compensating benefit from more finished games.

The cheap moves were not free. With a quarter of moves searched to 600 visits and the rest to 150, the cheap moves still consumed 42.9% of nominal search work. Nor were they isolated from training: they selected played moves, affected terminal outcomes and retained trees, could supervise a preceding row's next-policy target, and influenced which positions entered the restart archive. "Not stored as a primary policy row" was never the same as "irrelevant."

The mixed workload also interacted badly with batching. Once the cheap searches finished, only the full-search minority remained. In a 512-game test this tail left about 128 active trees and filled only 86 of 320 available batch slots with one leaf per tree. Allowing four leaves per tree raised the average batch to about 268 and improved search throughput by roughly 20%, but it did so by making each search less serial. A policy intended to save compute thus created pressure to accept a search-quality tradeoff merely to keep the accelerator occupied ([batch-fill study](#)).

Most importantly, the design exposed a semantic failure at the end of games. Forcing cheap searches after a late ply removed searched endgame rows, while using the shallow cutoff value could propagate a weak soft target back through the capped game. The later cut-game repair and removal of the cheap tail addressed that failure ([cut-game target study](#)). The project therefore kept KataGo's broader insight—that search cost and target eligibility are distinct choices—but superseded the random fast/full recipe with a full search for every recorded move. This was a systems-and-target-semantics decision, not a clean matched-compute Elo victory for all-full search; no such long-run ablation was preserved.

Three attempts to allocate search adaptively

The first attempt asked whether search could stop when the visit leader looked uncatchable. An audit of completed trees found apparent room: an optimistic reconstruction suggested that a rule requiring at least 75% of the cheap budget might remove 15.3% of cheap-search visits, or 6.38% of all nominal limits if full searches saved nothing.

But the records contained final visit distributions, not the intermediate traces needed to identify when the leader first became safe. They also omitted enough state to reconstruct retained visits and forced-playout-pruned targets. The apparent saving was therefore an oracle-like opportunity estimate, not an observed stopping result.

A wider offline study found a deeper problem. The marginal value of search was not monotone in visit concentration: very diffuse and already-decided positions gained little, while moderately concentrated but contested positions gained most. Simple concentration thresholds tended to stop in the very region where more search was useful. The rule-based approach was audited and declined before an online strength match; it should not be described as an implemented algorithm that lost Elo ([termination audit](#)).

The next allocator replaced the rule with a learned prediction. An auxiliary head estimated, for several candidate budgets, the divergence between that budget's policy and a deep-search policy. A calibrated corrector incorporated root observables, and a dual variable kept average spend near its target. Deep labels, replay persistence, model publication, native budget selection, safety gates, and telemetry were all implemented. Mechanically, the system worked. At approximately matched mean spend it captured about 23% of the available policy-divergence headroom; its mean target fidelity resembled roughly 1.18 times uniform search at 0.967 times the spend. It also behaved plausibly, assigning more work to contested positions.

Learning nevertheless deteriorated. Repeated online attempts trailed comparable non-adaptive training by roughly 60–100 ladder Elo. More than a third of positions received an average budget fraction near 0.36, and almost 9% received one eighth of baseline search. Those shallow policies were close to the network's own prior yet were trained at full weight. The likely failure was therefore objective mismatch: closeness to a deep policy at the current position does not measure how much a target will improve the next network. That mediation remains a hypothesis, but the central result does not: the controller improved its stated proxy and still produced worse learning ([analysis, frozen-trunk probe](#)).

The final adaptive system moved the decision inside search, where a learned stopper could observe the evolving tree rather than predict difficulty in advance. Its decisive test started from the same checkpoint, optimizer, and rebuilt replay state for every arm. The most aggressive setting skipped about 14% of nominal search, and its internal credit-wait measurements changed in the expected direction. Yet generation cadence improved by only about 3%. Self-play overlapped the optimizer, so most of the removed search was slack rather than critical-path work. Paired strength estimates were $+1.7 \pm 9.9$ Elo and -4.2 ± 10.1 Elo for the two stopping settings: neither resolved a strength effect. At the observed learning rate, the cadence gain was worth only about one Elo over three hours, below the experiment's resolution ([shared-state conclusion](#)).

These studies failed for different reasons. The threshold rule lacked an identifiable safe signal. The predicted allocator optimized a measurable but insufficient proxy. Learned stopping saved the resource it targeted, but that

resource was mostly off the critical path. In a non-overlapped or inference-bound system the last conclusion could change; nothing here proves adaptive search universally useless.

Parallel leaves: buying latency with search quality

Batching across many independent games is the cleanest way to feed the accelerator, but the number of active roots eventually runs out. The engine can then keep several leaf traversals in flight from one root. Virtual reservations discourage those traversals from selecting the same path, yet every selection is based on a tree that is missing the other in-flight results. Parallel search is therefore not serial MCTS executed faster. It exchanges fresher decisions for larger batches and lower latency.

The experiments exposed how easy it is to measure the wrong regime. An initial sweep suggested little cost per doubling, but its batch capacity was too small relative to the number of trees, so the configured per-root parallelism barely bound. A deliberately oversized-batch rerun activated the knob and estimated a steeper -6.4 ± 4.7 Elo per doubling. The interval still included a small effect, and the rerun was diagnostic rather than a production throughput configuration ([initial sweep](#), [binding rerun](#)).

The tradeoff also depended on the total budget. At 1,000 searches, a terminal fixed-network sweep measured point losses of 19 Elo with four leaves and 45 Elo with sixteen leaves relative to serial search, while sixteen-way parallelism was substantially more damaging at 100 searches. The likely explanation is that a deep search will eventually visit more of the temporarily suboptimal leaves selected from stale state. However, only a few budget and parallelism combinations were measured, and the terminal strength curve itself uses different parallel counts at different depths. It is not a pure scaling curve from which a universal safe-parallelism law can be inferred ([terminal evidence](#)).

The retained design consequently uses bounded parallelism as a serving parameter, not a free algorithmic speedup. The right setting depends on available independent roots, batch capacity, total search depth, latency requirements, CPU load, and memory. Older results from the fast/full batching tail do not directly prescribe the topology of the final all-full workload.

When a tree became a graph

Chess appears rich in transpositions: different move orders often reach the same board. The project tested whether Monte Carlo graph search could turn those transpositions into shared neural evaluations, descendants, and search statistics. This was a complete implementation, not a cache mislabeled as graph search. Canonical nodes held shared state-level information; parent/action edges retained local PUCT statistics; correction backups exposed better shared values to incoming edges; and the system handled trajectory reservations, cycles, rerooting, pruning, and capacity reclamation. An audit against both the paper and its reference implementation found and corrected a missing first-link backup behavior.

The limiting fact was chess-state identity. Pieces and side to move are not enough: castling rights, en-passant state, the halfmove clock, and repetition-relevant history can change the legal result. Merging positions that differ on

those fields would create an approximate algorithm with different game semantics. Under exact equality, most apparent board transpositions disappeared.

At ordinary budgets, verified links were absent or negligible. After the first-link correction, only 0.0249% and 0.1769% of neural evaluations were avoided at 1,000 and 10,000 searches, while the graph was 8.63% and 8.28% slower. At 30,000 and 60,000 searches, table hits rose to 2.37% and 3.46%, but avoided evaluations remained approximately 0.0001% and 0.0348%; throughput was still 7.06% and 5.76% lower. Structural counters confirmed that shared descendants and statistics were active. A final strength match was unnecessary to answer the deployment question: exact reuse was orders of magnitude too sparse to repay the bookkeeping cost. The corrected implementation and its negative result are preserved with the [graph-search release](#), including the [paper audit](#).

Why inference caching found little to reuse

Inference caching asked a narrower question: if an identical encoded neural input reappears under the same model, can its policy and value outputs be reused without sharing search state? Two investigations answered complementary parts of it.

First, a real bounded, sharded cache was shared by the search threads within each self-play process. With eight processes per GPU, three search threads per process, and capacity for 1.5 million entries per process, its hit rate was 0.970%. Disabling the cache made game updates about 0.88% faster in the short stochastic comparison and reduced summed worker peak memory by about 7.12%. The implemented cache consumed memory without demonstrating a speedup ([implemented-cache benchmark](#)).

A later audit measured the upper bound available to a wider cache before building one. An unbounded tracker shared across one search executor observed exact encoded inputs but deliberately evaluated every position. With diverse starts and production-style progression, repeats were 1.33% at 150 searches, about 4.24% at 800, and about 3.5% in the mixed workload; same-batch duplicates were essentially absent. Following retained trees for several moves raised the rate only to roughly 4%. The tracker itself cost about 3.65% throughput and grew without bound. A real cache would additionally pay for output storage, synchronization, eviction, and device transfers while retaining fewer entries.

The wider design was declined before implementation; it was an opportunity audit, not a production cache benchmark. The preserved [opportunity audit](#) and the earlier implementation together support the decision more strongly than either alone.

The retained search design

The selected system returned to fixed visits, but not to a bare or naive search. It uses PUCT, reduced-parent-value first-play urgency, root noise and temperature for self-play exploration, forced root payouts followed by pruning of visits that should not become policy supervision, batched native inference, a 0.99 per-ply value discount, and tree retention with 60% of visits carried across a played move. Every recorded move receives the current staged visit cap. The authoritative settings live in the [final chess configuration](#); the current mechanics are described in [Search and self-play](#).

Those ingredients do not form an additive ablation table. Search depth has direct strength and target-fidelity evidence. Evaluation and self-play clearly need consistent first-play-urgency semantics. Parallel leaves have a measured, workload-dependent quality/latency tradeoff. Root retention, forced playouts, noise, discount, and the exact PUCT constants are retained parts of a coherent AlphaZero-style recipe, but most lack isolated final-workload strength estimates.

The overall conclusion is correspondingly narrow. For this diverse chess self-play workload, fixed budgets were strong, legible, and operationally robust. The adaptive systems either lacked a safe observable, optimized the wrong proxy, or removed work that did not control elapsed training time. Exact graph and cache reuse were too sparse to cover their overhead. Different games, a repetitive analysis service, or a non-overlapped learner could reverse some of those economics. They do not reverse the decision supported here: under the measured workload, the simplest auditable allocation was the most dependable one.

4B. Getting more learning from each game

Self-play positions are not interchangeable units of data. Some contain a large correction from search; some repeat an opening the model already understands; some carry a reliable terminal result; and some end only because the runtime reached a wall-clock-motivated cap. The replay system decides which of those positions survive, how often they return to the optimizer, and which parts of each target are actually known. Those decisions shape the learning problem as directly as the network or optimizer does.

The project gradually separated four operations that are easy to confuse. *Generation* chooses the trajectories to search. *Admission* turns eligible observations into durable rows. *Selection* chooses rows for a batch. *Weighting* changes their contribution after selection. A fifth mechanism, presentation credit, governs when training may advance. This vocabulary matters because a technique that prioritizes future games is not the same as prioritized replay, and drawing a row more often is not the same as increasing its loss weight. The underlying investigations are indexed in the [data and replay experiment ledger](#).

Replay is both memory and clock

A replay window mediates between freshness and breadth. A small FIFO quickly removes targets made by weak old policies, but concentrates learning on a narrow recent distribution. A large window preserves more openings, endgames, and policy eras, yet can keep stale targets alive after the acting model has moved on. Merely allocating a large capacity does not create information: early in training, a twenty-million-row store may contain only a small fraction of that amount.

The retained design therefore preallocates the physical memory map once while growing its logical capacity through 0.6, 1.2, 2.0, 2.8, 4, 6, 8, 12, 16, and 20 million rows. Growth avoids pretending that an empty early window offers diversity and later allows a broader policy history to remain available. A mature earlier campaign showed why freshness must be measured rather than inferred: strength continued to improve after fixed-dataset policy accuracy largely saturated, while larger models and deeper search reduced the rate of new positions. That

observation motivates the wider window but does not isolate this exact schedule as a strength improvement ([training-dynamics audit](#)).

Replay reuse introduces a second tradeoff. The configured ratio is the number of optimizer presentations funded by each newly admitted row. In the retained setting, four presentations are credited per row; a 500-step quantum at a global batch of 2,048 therefore requires 256,000 newly appended rows. Credits are issued only after append and flush, and a persistent ledger prevents a restart from spending them twice.

This ratio is not merely a sample-efficiency knob. Because training, evaluation, model publication, visit schedules, and replay growth advance at quantum boundaries, changing reuse also changes their wall-clock pace. Higher reuse funds more optimizer work from the same generated data; lower reuse exposes each update to more fresh positions if the actors can supply them. Short controls at ratios four, 6.25, and eight remained matched at their shared evaluation boundaries despite different update rates, but all ended within roughly 90 minutes and their full raw curves are not preserved in the benchmark archive. The completed campaign also changed replay capacity, optimizer, objective weighting, and inference alongside reuse. Ratio four is consequently a deliberate freshness bias, not a measured standalone explanation for final strength. Both configured reuse and effective presentations per distinct admitted row are needed to describe what the learner actually saw.

Choosing information without inventing it

Within the live window, the sampler asks where search most revised the raw policy. Policy surprise is the divergence between the visit distribution and the network prior. Seventy percent of draws are allocated in proportion to this signal, capped at 2.0, while 30% remain uniform. The cap keeps a few extreme rows from monopolizing training and the uniform component preserves broad coverage. The same row may reappear across optimizer steps, but sampling is without replacement within one global batch.

This is prioritization, not an unbiased estimate of the uniform-replay objective. The implementation deliberately does not apply inverse-probability correction, so the optimizer learns from the prioritized distribution. Surprise can also reflect search noise, target age, phase of play, or the number of legal moves—not only genuine difficulty. The sampler is part of the successful retained bundle, but there is no isolated online ablation for its 70/30 mix or cap.

Loss weighting is a separate path. Each row has a typed positive sample weight which, after batch-mean normalization, multiplies the primary and eligible auxiliary losses. Ordinary rows in the selected recipe use weight 1.0. Historical replay could aggregate duplicate positions and encode multiplicity through weights; that verifies that the mechanism worked, not that arbitrary weighting improved play. Claims that the final system “weights hard positions more” should therefore refer to draw probability unless a non-unit loss-weight schedule is explicitly being discussed. TD-error priority, recency weighting, and global deduplication remained proposals rather than completed current experiments ([sample-stream audit](#)).

Admission happens earlier still. Random opening-prefix moves and the reconstructed prefix of a restart game have no search observation and create no replay row. A normal searched move does. A final search performed only to value a cut position can also become a row even though it selects no played action. Sparse policy storage retains at most 60 actions and records how much visit mass was discarded. These rules define the data set before sampling or weighting can act on it ([replay system](#)).

Starting games where information is likely

The retained start mixture attacks two different coverage problems. Half of games begin after a uniformly selected zero to eight random legal plies. These prefixes cheaply diversify the opening without pretending to be a balanced opening book. They are reconstructed in the recorded history but not searched or trained directly; their value is the downstream positions they expose. Drawing zero plies naturally retains some games from the ordinary initial position.

The other half begin from archived self-play states. A position is eligible only if at least 15 plies remained in its source game, its absolute root value was at most 0.8, and two or three leading actions covered 85% of visit mass. The branch actually played is marked used. A later restarted game reserves one untried plausible alternative and forces that action after reconstructing the prefix, so new compute explores a branch the source game did not.

Selection from this archive gives 30% probability to uniform choice. The remaining reservations favor the square root of *value correction*: half the absolute difference between the searched root value and the raw network value. The square root softens the priority so one extreme correction cannot dominate. Age and capacity bounds remove old states, reservations prevent duplicate claims within a worker, and exhausted positions leave the archive. This is the project's practical difficult-state curriculum, but "difficult" is an interpretation of disagreement, not a ground-truth label. The mechanism does not train a regret network, and archives are local to workers rather than globally deduplicated.

Restart priority and replay surprise are related only in motivation. One changes which future trajectories are generated; the other changes which already stored rows are selected for optimization. Their individual Elo effects, the 50% start mixture, and the exact filters were not isolated. They should be presented as a coherent curriculum design grounded in branch coverage rather than as independent measured gains ([reference-recipe analysis](#)).

Ending games without corrupting their labels

Resignation can save a large amount of search, but a false resignation turns a drawable or winning position into an incorrect terminal label. A fixed value threshold was therefore replaced by continuing measurement. Twenty percent of games are designated at creation as continuation games and can never resign. They reveal the outcomes of hypothetical triggers for thresholds from -0.99 through -0.70. A trigger requires both the root value and the best visited child's backed-up value to cross the threshold; evidence from capped games is excluded because their natural outcome remains unknown.

Over a rolling window, a candidate threshold needs at least 100 triggered continuation games and a one-sided 95% upper confidence bound on false non-loss no greater than 2.5%. The threshold may become more conservative immediately but relaxes by at most 0.01 per publication. An intentionally aggressive canary verified trigger journaling, persistence, and exclusion of a capped continuation, but did not establish a safe production threshold ([resignation canary](#)). The defensible conclusion is that resignation is calibrated and continuously audited—not that it has zero error or a measured independent Elo gain.

A ply cap creates a harder target problem because there is no observed result at all. The project compared material heuristics, raw network value, values from adjacent searches, and a fresh search at the cut position. Playing games beyond the normal cap supplied later outcomes for evaluation. At the earlier measured checkpoint, the cut-position search achieved Brier score 0.444 and cross-entropy 0.756, compared with 0.491 and 0.851 for material divided by 39. At the later checkpoint the corresponding scores were 0.193 and 0.374 versus 0.388 and 0.707. Search-root sign accuracy exceeded 98% in both cohorts; calibration, not merely sign, distinguished the targets. The retained worker therefore performs one full search at the actual cut position and uses its root value as the bootstrap ([cut-position benchmark](#)).

The cut policy also closes the most damaging data failure found in the project. A former cheap-search tail removed searched endgame rows while broadcasting one shallow cutoff estimate back through each affected game. Weak endgame targets then produced weak conversion, more capped games, and more weak targets; replay eviction could remove the bad rows while their effect remained in the weights. The full reconstruction, measured incidence, and two-stage repair belong to [the late-game target-poisoning study](#). Here the relevant data rule is simply that cut-value provenance and policy-row eligibility must be audited together.

Knowing what each target means

Three value mechanisms act at different boundaries and should not be collapsed into "value discount." During materialization, the terminal WDL target is blurred toward uniform by 0.998 for each remaining game ply. During optimization, a scheduled coefficient eventually blends up to 10% of the stored search-root scalar into that discounted outcome. Inside search, a separate 0.99 per-tree-ply discount changes backups, selection, and the root value before replay is written. The cut-position benchmark supports searched values as better bootstraps than the tested material heuristic; it does not isolate the ordinary 10% blend or either discount as a strength gain.

Future-dependent auxiliary targets require equally explicit eligibility. The retained next-policy target uses the sparse visit distribution from the following searched observation; it is unavailable if that observation does not exist. Remaining game length is known only when the game reaches a natural result or valid resignation, so every row from a cut game marks that target ineligible. The objective masks and renormalizes auxiliary losses over eligible rows rather than treating missing labels as zeros. Overfit tests established that the supplied multi-head targets were learnable, while replay audits established that censoring

and offsets were wired correctly. Neither establishes an independent playing-strength gain ([multi-head overfit study](#)).

Broader auxiliary bundles were removed during a period with several simultaneous training problems. That was a precautionary simplification, not a controlled negative ablation. Retaining next-policy and remaining length later does not prove that either head helped, just as removing the others does not prove that they harmed. The firm result is semantic: labels derived from the future must be materialized from complete trajectories and masked when the future is unknown.

Fresh targets versus fresh positions

An older replay design synchronously re-searched some recent positions and stored source-bound target overrides. No controlled study established that this used search more effectively than generating fresh states and outcomes, and the mechanism disappeared with its replay schema. Reanalysis is therefore superseded infrastructure, not a negative efficacy result ([historical implementation record](#)).

Model publication addresses freshness from the other side. Publishing every 100 optimizer steps was implemented historically and rejected because repeated serialization, deployment preparation, validation, synchronization, and activation outweighed the observed benefit. The raw timing bundle and matched strength curve were not preserved, so this remains a qualitative decision record, not an effect-size result. The retained 500-step boundary is a compromise, not a universal optimum; its cost and freshness also depend on replay reuse ([publication-cadence record](#), [current decomposition](#)).

Overlap without full asynchrony

Training proceeds in coordinated credit-funded quanta while half the actor processes continue self-play. A pause sweep found only a narrow, workload-dependent difference between balanced choices, so the retained fraction is an operational setting rather than a universal optimum. Publication, replay snapshots, and optimizer commits remain synchronized; fully asynchronous learning was proposed but not implemented. The throughput measurements and a corrected device-placement artifact are covered in [Chapter 5](#) and the [pause benchmark](#).

Making the data trustworthy

Completed games pass through bounded, worker-owned materialization into a typed circular store. Atomic writes, quarantine with a fatal rejection ceiling, immutable trainer snapshots, deterministic cross-rank sampling, and credit only after durable append preserve the meaning of the experiments under load. These are correctness and

throughput properties, not direct evidence of stronger chess; the implementation and benchmarks belong to [Chapter 5](#), the [pipeline design](#), and the [materialization design](#).

What the retained curriculum establishes

The final data recipe combines staged replay growth, reuse four, policy-surprise sampling with a uniform component, unit row weights, shallow random openings, branch-reserved restart states, calibrated resignation, direct cut-position bootstraps, explicit target eligibility, coordinated publication, and partial actor/trainer overlap. Most of these choices coexist in one successful system and do not have one-variable online Elo estimates.

The strongest direct comparison favors a searched value over the tested material heuristic at a cut position. The strongest incident evidence is the late-game poisoning chain. The clearest systems lesson is that more presentations are not necessarily more information: replay reuse, target freshness, state diversity, and schedule pace move together. The retained design spends compute on a broad, inspectable stream of searched positions and preserves the meaning of every admitted target. It should be understood as an integrated curriculum, not a collection of additive strength bonuses.

4C. Choosing what the network predicts

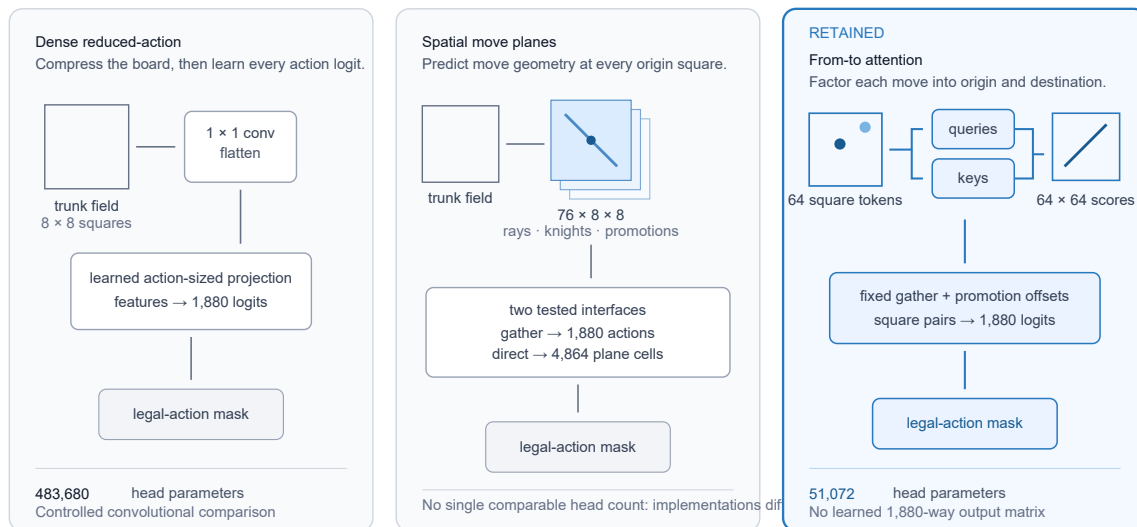
The network had to satisfy two objectives that are easy to conflate. It had to learn useful chess representations from self-play, and it had to evaluate thousands of search leaves cheaply enough that those representations improved within the available wall-clock budget. The resulting design was not selected by parameter count alone. Policy representation, trunk structure, auxiliary supervision, quantization, and model growth were all tested against the same end-to-end constraint.

Every model receives a 52-plane, side-to-move-canonical chess representation. The planes include pieces, castling rights, en passant, checks, repetition state, the eight most recent moves, material counts, and the fifty-move counter. File reflection is the only training augmentation; it mirrors the action targets and exchanges kingside and queenside castling planes. These rule-sensitive inputs are necessary to keep positions with different legal or draw states distinguishable, but their individual strength contributions were not ablated.

Three policy representations

The chess interface defines 1,880 canonical actions, but an action table does not determine how a network should predict them. The project implemented three materially different policy families.

Three ways to predict a chess move
All paths supervise the same legal search policy; the learned output geometry is what changes.



Schematic, not to scale. Parameter counts use the same controlled convolutional trunk.

Figure 4C.1 — Policy representation changes the learned output geometry. Dense heads learn an action-sized projection, move-plane heads preserve spatial move types, and the retained from-to head scores square pairs before a fixed gather. The parameter counts are from the same controlled convolutional-trunk comparison; no count is shown for the plane family because its implementations and external interfaces differed. The diagram is architectural, not a playing-strength comparison. Source: [attention viability study](#).

The first used a conventional dense reduced-action head: a small spatial projection was flattened and mapped directly to the 1,880 logits. Two-, four-, and eight-channel projections were explored, along with spatial reduction and low-rank final maps. A rank-96 variant reduced a roughly 484,000-parameter head to about 207,000 parameters while reportedly matching its immediate baseline. The underlying result bundle has not been recovered, however, so the individual dense variants cannot support a reconstructed quantitative ranking. Dense heads nevertheless trained successful models and should be described as superseded, not disproven.

The second family represented moves spatially. Its 76 planes comprised 56 sliding directions, eight knight moves, and 12 promotions. One implementation gathered the canonical action logits from this tensor; another exposed all 4,864 plane-square cells as the action interface and normalized only the legal cells. The native mapping was checked over 83,651 moves without a discrepancy, and controlled inference showed that the larger interface itself was not the source of the observed slowdown. A repaired convolutional plane head reached 2.1828 held-out policy cross-entropy after roughly 2,500 supervised steps, compared with 2.0824 for the dense control, while still improving more quickly. That shortened comparison did not establish convergence. The owner also recalls a plane-policy model learning more slowly and being retired after online self-play, but the corresponding result artifact—and even the recalled plane count—has not been identified. The structured-plane family is therefore technically validated but empirically underdetermined, not a quantified negative result.

The retained from-to head preserves square structure without predicting a mostly empty plane tensor. It projects

the 64 trunk squares into query and key vectors, scores all origin-destination pairs, and gathers the canonical actions through a fixed table. A separate projection adds queen, rook, and bishop offsets for promotions; en passant and the canonical castling encoding remain ordinary square pairs. On the controlled convolutional trunk, this head used 51,072 parameters rather than 483,680 for the dense alternative.

Holding that trunk fixed, the from-to head improved the held-out policy gap by 0.0298 nats, with a paired 95% interval of 0.0285--0.0311. Widening the trunk after recovering those parameters added only 0.0018 nats in the three measured cells, although the missing fourth cell prevents a full factorial conclusion. The serving cost was modest at production scale: approximately 1.9% of batch-512 forward throughput and 9% at batch 64. These measurements made the from-to head the best-supported policy choice, but they came from a shortened, single-seed teacher-data study, not an isolated long self-play match. Cross-entropy differences are reported as such and are not converted into an invented Elo gain. The full comparison and its dataset limitations are preserved in the [attention viability study](#).

The owner remembers roughly ten policy-head comparisons, including an online plane-head trial that trained more slowly and underperformed. The full result bundle has not been recovered; the preserved plane implementation has 76 planes while the recollection names 96. That result remains qualitative and the two descriptions should not be silently equated.

Convolution, attention, and global context

The retained trunk is convolutional. Its residual tower shares one feature field among the policy, value, and

training-only heads. Full sharing was an invariant throughout the implemented research programme; no split policy/value trunk was tested. The earlier impression that such a comparison existed arose from oversized policy heads, including a second policy-shaped auxiliary, consuming much of a small model's capacity. Head capacity and trunk separation are different questions.

Pure attention trunks were implemented with 64 square tokens, learned row and column embeddings, pre-normalized self-attention, and GELU feed-forward blocks. Packed query/key/value projection replaced the generic attention module. No-bias, learned relative-offset, and input-dependent Smolgen-style attention biases were implemented, though only the no-bias and Smolgen choices received a preserved efficacy comparison. A CNN-attention hybrid was proposed but not built.

Early comparisons were invalidated by different generation-zero policy shapes and by runtime and precision confounds. After bootstrap calibration and a matched policy head repaired the comparison, the convolutional model beat bare attention by 0.0060 nats on held-out teacher data. Smolgen produced the best attention cell, but much of the apparent improvement attributed to attention actually came from replacing its dense policy head: on the attention trunk, the from-to head improved the held-out gap by 0.1573 nats. Attention also used substantially more memory and served more slowly in the relevant batches. The project therefore retained convolution for this workload; it did not establish that attention is generally unsuitable for chess.

Local convolution is supplemented by a global-pooling residual module every second block. After the first convolution, one quarter of the channels supply board-wide means and maxima that are projected back as biases on the local features. Squeeze-excitation was also implemented and used historically. The owner recalls global pooling learning faster without a clear difference at convergence, but no result artifact has been recovered. Its retention is motivated by system experience and external KataGo evidence, not by a standalone chess Elo claim. It also leaves floating-point islands in the quantized graph, illustrating that useful context and serving efficiency were not always aligned.

Value and training-only heads

The outcome head predicts win, draw, and loss rather than a single scalar. Search converts this distribution to an expected value when necessary, while training and diagnostics retain the draw probability. A compact two-channel spatial reduction and 48-unit hidden layer was retained. A matched 32-channel probe added 97,020 parameters and reduced measured training throughput by 1.31%, while improving total loss by only 0.00309 in one short seed and slightly worsening WDL loss. This justified keeping the smaller head, not a claim that its capacity is universally optimal. Likewise, the earlier scalar-to-WDL transition has no preserved isolated strength comparison.

The final training graph also predicts the later player's policy at a configured ply offset and normalized remaining game length. Their loss weights are 0.15 and 0.1 respectively. The next-policy target lives in the future state's own side-to-move action space and receives its own legality and symmetry transformations. Both heads are

stripped from the serving artifact. Wiring, eligibility, masking, gradients, symmetry, and checkpoint behavior were tested, but no long matched online ablation isolates either objective. Legal-move prediction and several proposed search or board state auxiliaries were not retained. During debugging, auxiliaries were also removed precautionarily without being shown harmful; their final inclusion is an assembled-recipe choice rather than a causal Elo result.

Bootstrap and optimization controls

Initialization is part of self-play because the untrained network creates the first search priors and therefore the first replay targets. In an early architecture comparison, the attention policy placed only about 0.11 of its mass on the top three moves while the convolutional control was effectively one-hot. Neither extreme was meaningful chess knowledge, and their different concentrations changed the data each model generated. The retained bootstrap path uses architecture-appropriate initialization, deterministic construction, a small final policy projection, and calibration on real encoded positions toward a common policy shape. Calibration controls concentration; it does not make an initial policy knowledgeable. A separate audit also found that the configured random seed had not originally reached model construction, so supposedly matched arms began from different tensors. Corrected comparisons now treat seed propagation and bootstrap shape as reproducibility requirements rather than hyperparameter wins.

The optimizer changed alongside the quantized architecture. AdamW trained earlier successful models and was superseded rather than disproven. Frozen-replay screens showed that Nesterov SGD could train the quantization-aware network stably and helped select warm-up, learning-rate, and folding behavior. Delayed folding improved the historical pre-fold schedule, and higher post-fold rates improved short-horizon replay fitting within the tested range. The production schedule was not a literal copy of the best short screen: the authoritative model remains pre-fold until one million optimizer steps, and the deployment copy inherits the main linear learning rate. Frequent gradient clipping in several screens did not prevent learning, but neither did it establish that the threshold was optimal. These are controlled optimization diagnostics; online playing strength belongs to the complete trained system. The supporting records are the [SGD replay screen](#), [pre-fold factorial](#), and [post-fold sweep](#).

Quantization as an architectural constraint

Post-training quantization of the ordinary residual tower was fast and behaviorally unusable. Activation ranges grew from about 0.78 near the input to roughly 40--43 late in the network; depending on calibration, full-trunk INT8 preserved only 13.3--25.9% policy top-one agreement. Weight-only quantization was faithful but slower than TensorRT FP16. Quantization therefore became a network-design problem rather than a final export switch.

A scaled pre-activation block bounded the learning graph, but its normalization, clipping, residual scaling, and requantization boundaries fragmented the compiled TensorRT graph. It expanded an 83-layer FP16 graph to 330 layers and an INT8 graph to 470 layers, with many reformat, yet still failed fidelity. The retained scaled

post-activation block instead keeps the conventional sequence of convolution, normalization, capped activation, convolution, normalization, scaled residual addition, and capped activation. Activations are capped at six, and residual branches are scaled by the inverse square root of depth. The scale can be folded into the second convolution at export, which preserves more efficient compiler tactics.

This block learned normally under quantization-aware training. After continuation in the folded deployment topology, a production-sized smoke test reached roughly 135,000 INT8 positions per second, compared with 60,000 for TorchScript BF16 and 99,000 for TensorRT FP16. These are model-core rates, not end-to-end self-play rates. Folding only after training damaged agreement, and global context and the heads remain outside the INT8 trunk. The result is evidence that architecture and deployment had to be co-designed; it is not evidence that the scaled block plays better than an ordinary residual block in floating point. The experiments are documented in the [quantization salvage](#), [pre-activation screen](#), and [scaled post-activation screen](#).

Progressive model sizing

KataGo provided the precedent: start with a small, fast network, train the next size on the same data, and switch when it catches up ([Wu, Accelerating Self-Play Learning in Go](#)). Early in self-play, extra model capacity may contribute less than the additional searched games a small network can produce. The [throughput benchmark](#) supports that premise here, and the small-to-medium handoff worked repeatedly. There is no equal-cost fixed-size control, so its exact Elo-per-currency contribution remains unknown.

Candidate start follows a stage-specific searched-Elo plateau; promotion instead requires two passing paired matches against the active model. The former loss-based gate promoted a candidate that was about 270 Elo weaker because extra catch-up updates made its training loss incomparable. The [failure study](#) explains that correction; the [sizing guide](#) owns thresholds, scheduling, and restart details.

The larger stage remains unresolved. An independently initialized candidate needed substantial catch-up. Explicit function-preserving growth avoided relearning the medium model's function, but may also bias optimization toward its existing representation; that possible capacity cost was not measured. The limited grown-model continuation reached parity without a clear strength gain, so the reported model remains medium-sized. A matched-compute comparison of independent catch-up and growth would be needed to choose between them.

Distillation and compact models

Two earlier compression programmes asked related but distinct questions. Teacher-output imitation trained compact students on a legal-masked policy and WDL distribution from policy-only teacher play; no search generated those labels. Increasing the dataset from one million to six million positions mattered more than a small capacity sweep. The strongest 1.33-million-parameter student trailed its teacher by 176.1 Elo at 25 searches each and by 38.4 Elo when it received the measured shallow equal-compute allowance of 58 searches. At 250 searches

each the gap widened to 257.6 Elo. Deeper search amplified the better prior rather than washing out approximation error. Fixed-ply sampling gave the dataset only one side-to-move parity, and the teacher had a known conversion weakness, so these absolute gaps do not transfer to the final model.

Replay-target compression instead trained on sparse MCTS visits, outcomes, root values, and replay metadata from a frozen ten-million-row window. The selected 474,069-parameter student was 13.20 times smaller than its teacher. It trailed by 291.3 Elo at 64 searches each and by 166.2 Elo when its measured saturated serving advantage allowed 186 searches against 64. It reached statistical parity only at an equal-multiply-accumulate allowance of 850 searches, which ignored tree work, launches, and imperfect batching and was therefore not an equal-time result. The compact artifact was useful, but it did not replace direct training. Detailed protocols appear in the [teacher-output study](#) and [replay-target study](#).

A terminal compression check trained a 470,295-parameter student on the final 20-million-row replay buffer. At 10,000 searches it reached 2,683 conditional benchmark Elo after roughly 7.5 epochs and 2,697 after roughly 23 epochs. The 14-Elo central difference lay well inside the match intervals, while held-out policy loss had become nearly flat. Tripling passes over this fixed buffer therefore produced no measurable playing gain. The longer student reached 2,873 conditional Elo at 100,000 searches, but the point is unbracketed and is not a calibrated deep-search headline.

Together, the studies show that a small student can preserve substantial behavior and make a practical published artifact, but not that model size can be exchanged mechanically for more search. Realized search multipliers were far below parameter or arithmetic ratios, and the teacher advantage often grew with search depth. Distillation is a compression result, not a stage of the primary self-play algorithm.

Decision

The retained network combines the rule-complete 52-plane input, a shared convolutional trunk, periodic global context, the from-to policy head, a compact WDL head, and training-only next-policy and remaining-length objectives. Scaled post-activation blocks make the trunk compatible with quantization-aware deployment. Progressive sizing successfully exploited a small model before handing off to the medium model, while the value of the larger stage remains unresolved. This is one integrated design supported by component tests of different strength; the report does not assign isolated Elo gains where only proxy, throughput, or assembled-system evidence exists.

5. Systems optimization

The purpose of systems optimization in this project is not to maximize an isolated benchmark. It is to turn a fixed hardware budget into more useful self-play positions and, ultimately, more strength per wall-clock hour. That distinction matters because model throughput, search throughput, completed games, admitted replay positions, and optimizer progress can move by very different amounts.

Native ownership of the search loop

Python owns experiment configuration, process supervision, replay, training, evaluation, and artifact publication. C++ owns the latency-sensitive path: game rules, encoded positions, search trees, selection and backup, batched inference submission, and inference-result processing. Moving this loop into one native runtime removed Python message serialization and per-position dispatch from the dominant self-play path. The old [Python MCTS benchmark](#) is therefore a reference floor, not an alternative production implementation.

The native boundary also enables batching across many simultaneous games. Each self-play process advances 512 game roots, and requests from those roots share a preallocated asynchronous inference pipeline. Trees survive played moves so previous work can be retained rather than reconstructed. The final topology uses four actor processes per GPU, one inference worker per process, batches of at most 320 positions, and two outstanding batches. Those settings are a joint CPU, memory, batching, and latency choice; they should not be interpreted independently.

Batching and host-side submission

Large neural batches are useful only if the host can continuously assemble, submit, and consume them. The project therefore optimized the entire submission path rather than the network kernel alone: board encoding, persistent staging buffers, host-to-device copies, result processing, CUDA event handling, CUDA graph replay, process count, and the number of inference threads.

In a controlled 32-process self-play workload, the optimized path increased search throughput from 512,679 to 617,782 searches/s, a 20.5% gain, while reducing aggregate actor CPU consumption from 52.6 to 19.8 cores. Average inference batch size rose from 141 to 222 and the number of model calls fell by 24%. On the same node restricted to 24 CPU cores, throughput increased from 215,265 to 496,036 searches/s because reducing submission overhead allowed the previously starved GPUs to remain busy. These figures are specific to the tested network and host, but they show why CPU efficiency can be decisive even in a nominally GPU-bound workload ([submission benchmark](#)).

More processes are not automatically wasteful. With the TensorRT actor, reducing the topology from four to two processes per GPU lowered simulations/s by 39%, despite both configurations filling individual inference batches. Process concurrency was needed to overlap the work around inference and saturate the device ([actor decomposition](#)). Conversely, excessive inference threads duplicated CUDA contexts and divided available requests into smaller batches. One inference thread per actor was the measured operating point after submission became cheap.

Search parallelism is a separate batching lever. Multiple leaves from one root can fill a batch after other games finish, but virtual reservations make the search less serial and can reduce search quality. Chapter 4 therefore treats parallel search as an algorithmic quality-throughput trade, not a free systems optimization.

Inference runtimes and precision

TorchScript provided the first production-quality trimmed policy/WDL artifact and remains the bootstrap and fallback runtime. It is important to compare against that serving artifact rather than against eager PyTorch. In an eager diagnostic, `torch.compile` improved batch-64 model throughput by approximately 27–33%, but the corrected comparison showed that fused TorchScript was faster than the compiled eager path. Training compilation was also unfavorable in the relevant eight-GPU control: at the selected batch and contention level it reduced throughput by about 18% relative to eager execution, whereas `bfloat16 autocast` improved it by 9.2% ([inference controls](#), [training throughput](#)). `torch.compile` was therefore not used as the production inference compiler.

TensorRT provided the stronger serving path. A native FP16 engine reached 75,889 positions/s against 40,716 for the TorchScript BF16 control in one matched backend benchmark, a 1.86x ratio. Quantization-aware INT8 inference added a further 1.31x over the TensorRT FP16 engine for the tested quantization-oriented network ([native backend benchmark](#)). Matched production-topology comparisons measured smaller but still material INT8 gains that depended strongly on model shape: 14.4% for the smaller network and 39.1% for the medium network ([small-network control](#), [medium-network control](#)).

These gains required the model and runtime to be designed together. Post-training INT8 was fast but changed policy and value outputs too severely. The retained design uses scaled post-activation residual blocks trained with fake quantization, quantizes the backbone convolutions, and leaves the start block, policy and value heads, and linear layers at higher precision. Each published checkpoint is recalibrated, exported through an explicit Q/DQ ONNX graph, and refitted into shape-specific TensorRT templates. Fidelity is checked on encoded chess positions with legal-action masking, policy divergence, and WDL error; engine construction success is not treated as semantic validation. The relevant feasibility evidence is the [architecture screen](#) and [frozen-replay QAT study](#).

Model shape remained a systems variable even at similar parameter counts. Width and depth changed kernel efficiency, TensorRT tactics, and memory behavior discontinuously. Channels-last layout and cuDNN autotuning helped relevant CNN shapes, but no analytic parameter-count rule predicted the fastest network. Progressive model sizes were therefore benchmarked at their actual serving batch and precision rather than selected from FLOPs alone ([CNN throughput study](#)).

Replay materialization and training supply

Search work becomes useful training compute only after games finish and their observations pass replay admission. Completed games are atomically published, converted into typed columnar rows, and appended to a fixed-capacity memory-mapped replay. Materialization is partitioned before expensive conversion, bounds inbox scans and staging, quarantines malformed individual games, and fails loudly if systemic rejection prevents the learner from receiving data. This makes replay ingestion both a throughput stage and a scientific boundary.

The trainer reads vectorized batches from the memory-mapped store with pinned-memory prefetch and one persistent DDP rank per GPU. A loader benchmark on

2.5 million rows improved from 3,943 to 32,579 samples/s after compacting 5,000 small producer shards into 25 containers—8.26x faster and 44% above the measured trainer demand ([replay-loader benchmark](#)). In a later live interval, materialization appended 8,790 positions/s while accepted positions arrived at 1,365/s, leaving more than sixfold headroom. Materialization was not the bottleneck in that workload ([end-to-end decomposition](#)).

A credit ledger links optimizer work to newly admitted positions. Each training quantum becomes eligible only after enough materialized data has arrived for the configured replay-reuse ratio. This prevents rejected games, ingestion stalls, or an accidental change in data reuse from appearing as normal optimizer progress. Persistent DDP ranks then avoid repeated startup and model construction costs. On the measured eight-GPU topology, a global batch of 2,048, bfloat16 autocast, and concurrent self-play produced 6,252 training samples/s; larger batches improved hardware throughput further but would also change update count and optimization semantics, so they were not adopted merely because the GPU benchmark was faster.

Overlapping self-play and training

The runtime pipelines self-play, materialization, training, checkpoint publication, and evaluation, but it is not a fully asynchronous learner. Training still occurs in explicit optimizer quanta, and workers switch models only at a defined publication boundary. During a quantum, a selected subset of actors continues generating games while the others pause. This preserves a clear data and model identity while using otherwise idle capacity.

The overlap fraction is a measured compromise. With no actors running, the trainer processed 25,275 samples/s. Keeping 8, 16, or all 32 actors active reduced trainer throughput to 21,492, 17,139, and 9,210 samples/s, while simultaneously producing approximately 506,000, 606,000, and 742,000 searches/s. When both training time and the remaining self-play wait were included, the estimated complete-quantum times were 117.1, 112.9, and 111.2 seconds. The broad result was that moderate overlap improved total cadence, while running every actor nearly doubled the trainer quantum for only a small additional end-to-end gain. The retained half-active policy sits near that measured tradeoff rather than maximizing either trainer or actor throughput alone ([overlap benchmark](#)).

The best overlap also changes with search cost. When searches are cheap, training dominates and pausing policy has little effect. As visit budgets and model size grow, self-play becomes more expensive and additional concurrent actors become more valuable. A fixed topology should therefore be justified in the regime where it will be used, not extrapolated from the beginning of training.

From inference speed to learning speed

The central systems result is the gap between simulation throughput and replay throughput. In one production-shaped comparison, TensorRT INT8 executed 1.86 times as many simulations as the floating TorchScript control. In the live training interval, however, accepted replay positions improved by only 23.3% relative to the cited floating reference. The faster actor completed many more but shorter games, actor population was reduced

during training, checkpoint publication consumed time, and replay credit appeared only when a complete game passed materialization. The measured accepted-position rate nevertheless predicted the observed optimizer cadence exactly ([end-to-end decomposition](#)).

Systems results should therefore be reported as a chain:

1. model positions per second;
2. completed search simulations;
3. completed games and eligible observations;
4. admitted replay positions;
5. optimizer presentations and steps per hour;
6. strength gained per wall-clock hour.

An optimization is valuable to this project only if its effect survives far enough down that chain. Core inference and search benchmarks diagnose mechanisms, while the credit ledger and learning curve establish whether the saved compute became useful training.

Finally, absolute throughput is not portable across machines. CPU quota, GPU power limits, PCIe and NUMA layout, driver/runtime versions, model shape, batch fill, and concurrent training all affect the operating point. Node comparisons informed hardware selection, but quantitative claims in this chapter remain attached to the documented hardware and workload ([hardware comparison](#)).

Three failures that changed the method

Most of the project can be explained by comparing alternatives without reconstructing when each experiment ran. Three failures are different. In each case, the order of events is part of the mechanism: an apparently reasonable optimization changed the data or model that the next stage consumed, the resulting failure exposed an invalid assumption, and the repair changed the system's acceptance criteria.

Late-game target poisoning

The self-play design once reduced search near the end of long games. The intent was to keep noisy positions from exceptionally long endgames from dominating replay. If those positions would not become training targets, spending full-search compute on them seemed wasteful, so the tail used cheap searches instead. Only full-search positions were admitted as primary training rows, however, so the cheap-search tail selectively removed the conversion phase from the training set. The resulting policy was weakest exactly where it needed to turn an advantage into a terminal result. Games wandered to the ply cap more often, and the cutoff then assigned one terminal value to every earlier training row in the trajectory.

This created a closed feedback loop. Missing searched endgames produced weak late-game play; weak play produced drawn-out, unconverted games; the cutoff supplied an unreliable target; and that target reinforced the same policy and value errors. During the most affected early interval, roughly one ply in seven was structurally ineligible for training, 27--32% of games reached the cap, and about 36--38% of admitted rows inherited the value assigned to a cut game. The corruption was therefore not confined to the last position: the final target propagated back through the entire recorded trajectory. By the time the active replay buffer looked healthy, the damaging rows had

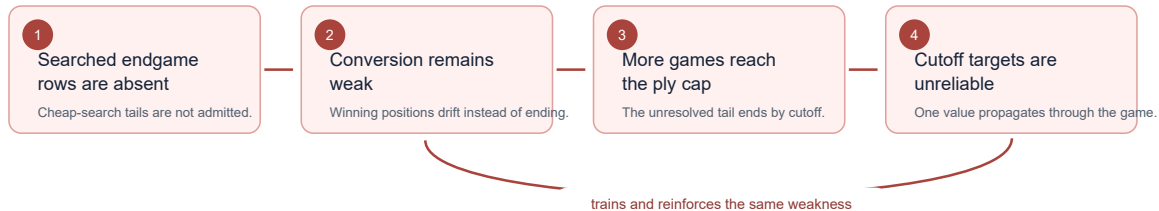
already been evicted, while their effect on the weights and subsequent self-play distribution could remain. The

reconstruction is documented in the [late-game training-data audit](#).

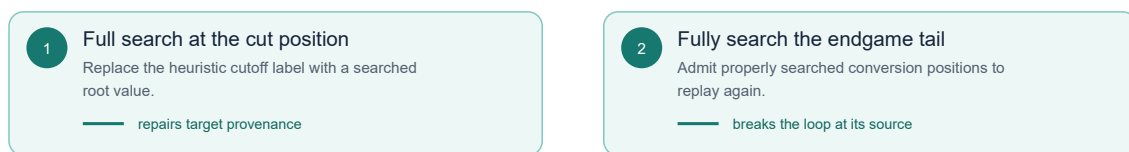
Late-game target poisoning

A search-saving tail became a self-reinforcing training-data defect.

FAILURE LOOP



TWO-STAGE REPAIR



Search eligibility, replay admission, and terminal-value provenance must be designed together.

Late-game target poisoning feedback loop and its two-stage repair

The cutoff repair fixes target provenance; restoring fully searched endgames breaks the feedback loop at its source.

The repair happened in two stages. First, the cutoff stopped relying on the material heuristic and obtained a value from one full search at the final cut position. A controlled continuation study supported that choice: on 2,282 early cut positions, the searched root value reduced Brier error from 0.491 to 0.444 and cross-entropy from 0.851 to 0.756 relative to the material target. The same ordering held on a later set of 1,144 positions. Later, the forced cheap-search tail was removed altogether, returning properly searched endgame positions to replay. The second change closed the data hole that a better terminal value alone could not repair. Because other late-game changes were bundled around the same period, neither step receives an isolated Elo credit. The evidence establishes the failure mechanism and target-quality improvement, not a one-variable strength estimate; see the [cut-value study](#).

The broader lesson is that search eligibility, replay admission, and terminal-value provenance are one coupled design. Saving compute by weakening or discarding a particular part of the trajectory can make the learner least competent in that region, after which bootstrapping from its own weak continuation turns an efficiency optimization into a self-reinforcing data defect.

A mechanically valid but semantically invalid TensorRT refit

The deployment pipeline builds a structural TensorRT template and refits it with each new quantized checkpoint. One template was built from a checkpoint whose clipped activation quantizers had many numerically equal scales. At high optimization levels, TensorRT optimized the engine around those equalities. Training later separated the scales, but the refit API still accepted every replacement weight, reported no missing weights, and returned success.

The resulting engine was structurally valid and semantically wrong.

The defect was visible only when the published engine was treated as a chess model rather than as a successfully serialized object. On 516 real positions, the faulty template produced legal-policy KL near 1.05 and repeated refits of the same inputs changed individual logits by 10–15. Building at a lower optimization level, or first making the template's scale constants distinct, reduced legal-policy KL to about 0.0012 and made repeated refits deterministic. The failure required three conditions: equal scales in the build source, optimization level four or higher, and a later refit that made those scales unequal. Tests with identity batch normalization and zero biases rejected constant folding as the cause. The complete reproduction and correction are preserved in the [TensorRT refit investigation](#).

Template construction now separates equal quantization scales before optimization and uses a safer default optimization level. More importantly, deployment publication no longer treats deserialization, complete weight accounting, or refit success as evidence of model equivalence. It evaluates the engine on real encoded positions and records legal-move top-one agreement, legal-policy KL, and WDL error against the source artifact. Those checks cover the semantics that search actually consumes: which legal move the policy prefers, how its probability mass changes, and whether the value distribution remains faithful. The publication boundary and retained measurements are described in [inference and evaluation](#).

The transferable lesson is simple: a compiler or refitter can satisfy its mechanical contract while violating the model's behavioral contract. Deployment artifacts must be validated as semantic models on representative inputs, and the exact graph, template, runtime, refit manifest, and probe results belong to the evidence for every published checkpoint.

Promotion from incomparable training losses

The progressive controller originally promoted a larger candidate when its smoothed training loss caught up with the active model. Loss looked like a cheap proxy for readiness: it was already measured during training, whereas a full-search candidate match consumed additional evaluation compute and time. That comparison ceased to be meaningful once the candidate received extra optimizer quanta over the same replay distribution. More presentations gave it a systematic advantage on replay loss without proving equal playing strength. The loss gate promoted a candidate whose deployed artifact passed inference-fidelity checks but was still about 270 Elo weaker in play.

Promotion now measures the property that publication requires. The candidate plays paired head-to-head matches against the active deployment artifact and must score at least 0.48 in two consecutive completed evaluations. A failing score resets the sequence; a failed or cancelled match contributes no evidence. Candidate-start timing, extra catch-up training, and promotion are separate controls. Function-preserving growth is likewise a separate attempt to remove the larger model's initial relearning deficit, not a substitute for the playing-strength gate. The

former loss rule, the current match rule, and the growth

Component Configured method

Compute	Eight RTX 4070 SUPER GPUs shared by self-play, training, and evaluation
Trainer	Eight-rank NCCL DDP; global batch 2,048; 500 optimizer steps per quantum; bfloat16
Optimization	Nesterov SGD, learning rate 0.1→0.01, momentum 0.9, weight decay 0.0001, gradient-norm cap 1.0
Deployment	Pre-fold INT8 QAT, real-position calibration, fixed-batch ONNX, TensorRT refitting
Network family	Scaled post-activation CNNs with global pooling, from-to policy, and WDL value head
Replay	Staged 0.6M→20M live rows, reuse ratio 4, policy-surprise sampling with 30% uniform mixing
Self-play search	Fixed staged visits, reduced-parent-value FPU, forced playouts, Dirichlet root noise
Starts	50% shallow random openings and 50% recent restart states
Targets	Policy and WDL plus next-policy and remaining-game-length auxiliaries
Evaluation	Paired openings against fixed-node Stockfish 13 anchors; native search for searched conditions

This table is explanatory. Exact schedules, paths, dimensions, and resource limits remain in the YAML.

Model, objective, and optimization

The configured ladder contains 12×128, 14×160, and 19×176 residual CNNs. Every stage uses capped scaled post-activation branches, global-pooling context in every second block, a key-size-128 chess from-to policy head, and a two-channel WDL value head with a 48-unit hidden layer. Training-only heads predict the next searched policy at weight 0.15 and remaining game length at weight 0.1; deployment removes both.

The primary policy and value losses each have weight 1.0. Terminal outcome targets are discounted by 0.998 per ply. A search-root-value blend rises from zero to 0.1 over its configured schedule, while search backup uses a separate 0.99 per-ply discount. Cut games use the searched root value and censor unknown remaining-length labels.

Each training quantum contains 500 optimizer steps. Eight ranks each process 256 positions for a global batch of 2,048. The optimizer is Nesterov SGD with momentum 0.9 and weight decay 0.0001; gradients are clipped to norm 1.0. The learning rate warms from zero to 0.1 over the first

procedure are specified in the [progressive-sizing promotion schedules](#).

Training loss remains valuable for optimization diagnostics. It is not a promotion criterion when the compared models have seen different numbers of presentations or when the artifact intended for deployment can be tested directly.

6. Integrated chess recipe

Living recipe and frozen result

The readable entry point for reproducing or extending the system is the fully expanded [chess-final-config.yaml](#). It owns the current network ladder, trainer, replay, self-play, deployment, and online-evaluation settings without an inheritance chain. It is a living recipe: future project work may revise it.

The reported experiment is immutable. Its selected checkpoint, model and engine hashes, evaluation rows, archive digests, and evidence status are recorded in the [final result](#) and [evidence index](#). Those frozen records—not a future state of the YAML—are the authority for published numbers.

Recipe at a glance

1,000 optimizer steps and then follows the configured linear schedule to 0.01. Training uses bfloat16 and persistent trainer processes; torch.compile is disabled.

INT8 quantization-aware training is active from the beginning. Calibration uses 516 real evaluation positions and is refreshed at every publication boundary. The trainable model remains pre-fold; deployment folding, recalibration, ONNX export, and TensorRT refitting operate on a copy. This keeps serving conversion from changing the optimizer's parameterization.

Self-play, replay, and curriculum

Self-play runs 32 actor processes, four assigned to each GPU, with 512 interleaved games per process. During a training quantum, half of the actor processes on every GPU pause while the others continue producing games. Native inference uses batch 320 with two outstanding batches per worker.

The fixed search budget grows from 300 to 800 visits per move. Search uses exploration constant 1.5, reduced-parent FPU with reduction 0.2, forced playout coefficient 1.5, Dirichlet epsilon 0.25, and alpha 0.3. Fixed visits were

retained after adaptive allocation and learned stopping failed to improve wall-clock strength.

Half of games begin after up to eight random legal plies. The other half use recent restart states filtered by value, remaining length, age, and branchable visit mass; restart selection retains a 30% uniform component. Game caps grow from 150 to 250 plies, and greedy move selection begins later as training matures. Calibrated resignation begins only after sufficient evidence, constrains the false-nonloss upper bound to 2.5%, and continues 20% of triggered games for ongoing safety measurement.

Replay capacity grows through ten stages from 600,000 to 20 million live rows. Each admitted position funds four training presentations. Sampling reserves 30% uniform probability and otherwise prioritizes bounded policy surprise. Eight materializers convert completed games into the fixed-layout memory-mapped store; credit is committed only against durable admitted data.

Progressive sizing

Candidate start and promotion are separate decisions. The primary 64-search ladder is smoothed and used to detect stage-specific plateaus; the full thresholds and window semantics are documented in [Progressive model sizing](#). An eligible successor receives an average of 1.5 optimizer quanta per active-model quantum on the captured replay snapshot and uses its own catch-up learning-rate clock.

Promotion is decided by candidate-versus-active matches, not training loss. The configured gate requires a score of at least 0.48 in two consecutive paired evaluations. This distinction matters because extra candidate presentations made the earlier loss comparison systematically favorable to the candidate and admitted a much weaker model.

The completed capacity study also grew the trained 14×160 network into 19×176 while preserving its function, then recovered deployment fidelity through QAT. The larger continuation reached parity but did not establish a stronger plateau. The reported checkpoint is therefore the 14×160 model. The limited continuation cannot distinguish insufficient training or post-growth optimization from a target, replay, or capacity limit.

Training-time and terminal evaluation

During training, evaluation ran every 20 minutes on 50 paired openings. The searched ladder used 64 searches per move against a three-rung adaptive bracket of fixed-node Stockfish 13 opponents; a policy-only ladder used the same opening and opponent framework. These measurements controlled progress and candidate timing but are distinct from the terminal result matrix.

The terminal matrix evaluated the selected 14×160 checkpoint over 100 games per row from 50 paired openings, with each opening played from both colours. Stockfish 13 used one thread, 1,024 MiB hash, and fixed node budgets from the published anchor curve. The reported rung for each model budget is the one whose observed score is closest to 0.5; both rungs and confidence intervals remain in the result record. Searched play used the frozen INT8 TensorRT artifact. Policy-only play used the matching float TorchScript export because it bypasses the native search service.

The measured curve spans policy only and 100, 1,000, 10,000, and 100,000 searches per move. Parallelism is one at 100 and 1,000 searches, four at 10,000, and sixteen at 100,000. It is therefore an attainable operating curve, not a single-variable search-scaling experiment. Absolute values are protocol-specific benchmark Elo and must not be presented as FIDE ratings or unrestricted engine ratings.

What the final result establishes

The terminal matches evaluate the assembled recipe. They do not allocate its strength gain among replay growth, restart states, auxiliary targets, resignation, progressive sizing, or any other bundled choice. Some components have isolated throughput, fidelity, proxy, or correctness evidence; fewer have controlled online strength ablations. The investigation chapters state those boundaries individually.

Likewise, the \$43.20 reported cost is 60 hours of accepted-lineage time at the recorded hourly price through the selected checkpoint. It excludes discarded work, later capacity experiments, distillation, terminal evaluation, and idle rental time. It is a reproducible denominator for the reported checkpoint, not total project expenditure.

7. Final training and evaluation results

Evidence state. Teacher training, the ten-row terminal matrix, the parallel-search sweep, and both distilled student experiments are complete and checksum-covered. The quantitative authority is the [final-run result record](#), backed by the [compact evidence index](#).

What was selected

The strongest fully retained checkpoint is a **6.32-million-parameter, 14-block, 160-channel convolutional model** with scaled post-activation residual blocks, global context conditioning, and a chess from-to attention policy head. It was trained with quantization-aware training. Searched results use its INT8 TensorRT artifact; policy-only results use the corresponding float TorchScript export because no native search service is involved. The inference path is therefore stated with each result rather than silently treating unlike backends as identical.

The ladder's strongest region spans several nearby checkpoints rather than a single isolated spike. Checkpoint 1026 was the last checkpoint in that region for which the full model, optimizer, QAT, ONNX, and TensorRT set had been retained. A subsequently grown 19-block, 176-channel model recovered its parent's strength but remained flat and did not justify replacing the selected checkpoint.

Training outcome

The report-scoped final lineage contains **180 observations through exactly 2.5 effective days**. It rose from 798 to **2,372.2** at the cutoff and reached a peak of **2,407.6** inside that interval. Later points belong to capacity and training experiments that did not improve the accepted result and are excluded from the comparison rather than presented as continued final-model training.

The previous four-day baseline is cut at exactly **3.0 days**, retaining 143 clean observations and ending at 2,265.4 Elo.

Later points from its noisy terminal evaluation interval are excluded. The two plotted endpoints differ by 106.8 Elo, and the final endpoint is 348.2 Elo above the early baseline's terminal observation. These are descriptions of the trimmed curves, not valid cross-campaign strength estimates, because the previous baseline used a single-rung fit while the final campaign's generic ladder series used a three-rung bracketed fit.

A retrospective plateau comparison on the same three-rung estimator places the previous baseline at 2,283.9 Elo and the final recipe at 2,358.0 Elo: **+74.1 Elo**, with an

approximately **± 15 Elo transfer/sensitivity allowance**. The allowance reflects uncertainty in transferring the estimator correction; it is not a game-level bootstrap confidence interval. The report therefore uses **about +74 Elo under a matched estimator** as the cross-campaign headline and does not compare raw peaks. Repeating the arithmetic with only the report figure's clean 3.0- and 2.5-day windows gives **+75.8 Elo**; the conclusion is not driven by the later points omitted from the plot. The [recomputable plateau table](#) retains both windows and the transfer assumption.

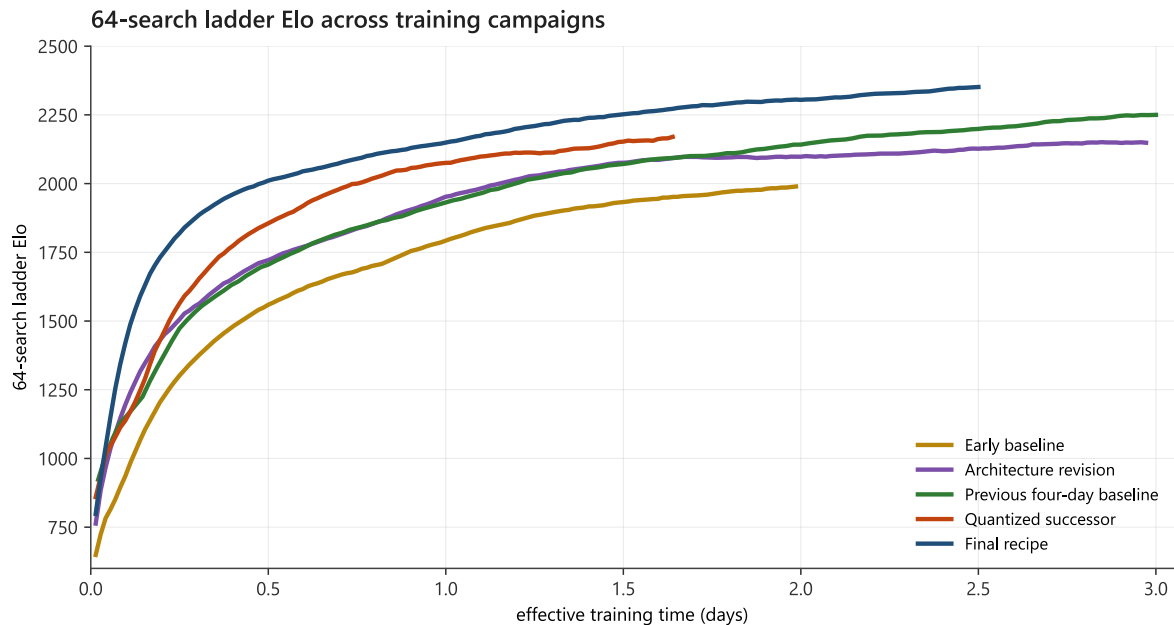


Figure 7.1: Each curve uses the bias-corrected 0.95 exponential moving average used by the project's earlier training-dynamics plots. The final recipe is cut at 2.5 days and the previous four-day baseline at 3.0 days. The tracked [publication input](#) records those rules and retains the source-series identities; internal run labels do not appear in the figure. The figure is descriptive; the +74.1 Elo claim comes from the matched-estimator plateau audit rather than from subtracting its displayed endpoints.

That stitched curve is not the entire compute history. It deliberately excludes two reverted branches while retaining their raw time in the source export:

1. An early INT8 conversion collapse was detected after one ladder observation and reverted.
2. A from-scratch capacity increase was promoted because training losses were compared across candidates that had seen different numbers of examples. The larger candidate was still roughly 270 Elo weaker. Training returned to the last valid checkpoint and removed 4,040,112 contaminated replay rows.

The second incident changed the method, not merely the operational state. Promotion now uses a match between the artifacts intended for deployment rather than training-loss parity, and the tested capacity-growth recovery starts from a function-preserving widening of the parent. The corrected larger model reached parity but did not break through the plateau during its limited continuation. This does not

identify the limiting factor: longer training, post-growth optimization, target quality, replay composition, and useful additional capacity remain confounded.

The selected checkpoint records **408,500 completed optimizer steps**. With the configured global batch of 2,048 this corresponds to **836,608,000 training presentations**. The frozen coordinator events cover all 817 contiguous training quanta on that path. Summing their per-quantum ingested-game counters gives **3,249,647 completed games**; the credit ledger ends at approximately **209.15 million net materialized positions**, or about **4.00 training presentations per net materialized position**. Replay held **16 million live rows** at the selected checkpoint. These are selected-lineage counters, not totals for every experiment or every game generated on the node. The cumulative position scalar is float32 in TensorBoard, so its final few integer digits are not meaningful. The [tracked trajectory and extraction method](#) preserve the boundary of each count.

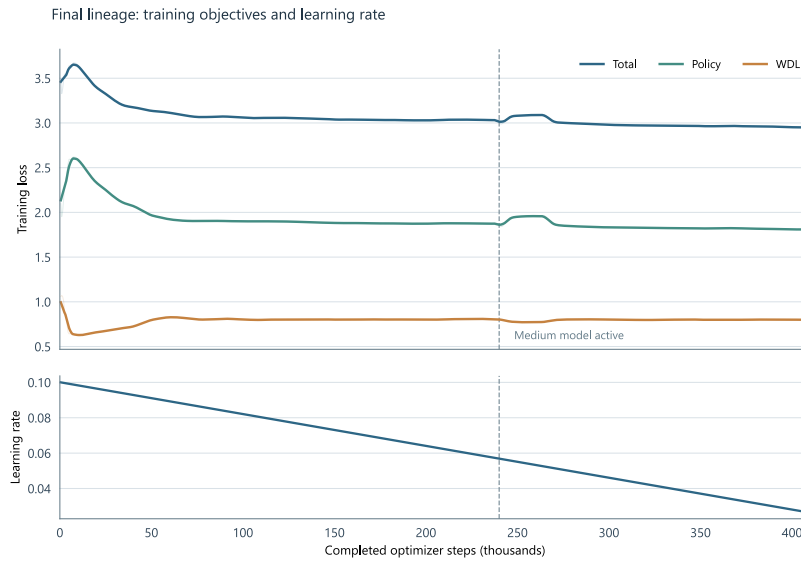


Figure 7.2: Total, policy, and WDL training losses are shown with an 11-quantum moving average over faint raw observations. The dashed line marks the small-to-medium active-model transition at 240,000 optimizer steps. Loss levels across different model stages are optimization diagnostics, not paired playing-strength measurements. The active learning rate fell from about 0.10 to 0.0266 by selection.

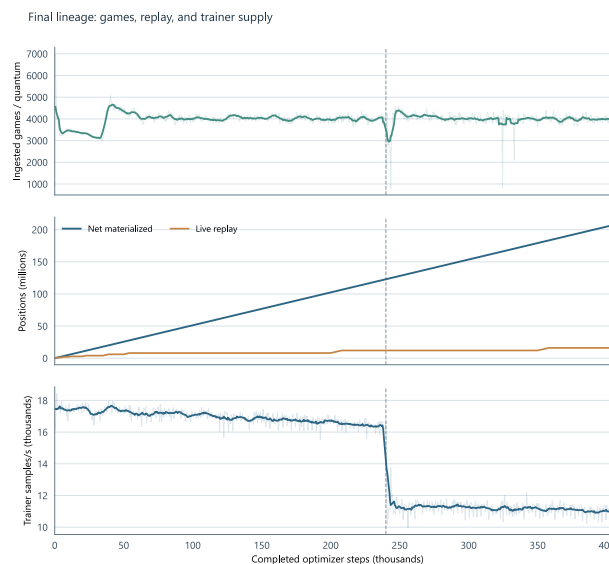


Figure 7.3: Per-quantum ingested completed games, net materialized positions, live replay occupancy, and training samples per second share an optimizer-step axis. The active small model's median measured trainer throughput was 16,977 samples/s across 480 quanta; the medium model's was 11,194 across 337 quanta. The stage comparison is descriptive: model shape, training schedule, and concurrent workload differ, so the gap is not an isolated model-size effect. The charts stop at the selected checkpoint and exclude later capacity experiments.

The narrow cost attached to the selected checkpoint is **\$43.20**, calculated as 60 accepted-lineage hours at \$0.72/h. It excludes the reverted work, later growth experiment, distillation, evaluation, and idle rental time. Until total spend is reconciled, it must not be described as the project's total compute cost.

Strength across four decades of search

The terminal protocol used 100 games per row from 50 colour-swapped opening pairs against single-threaded Stockfish 13 at fixed node limits. For each search budget, the reported rating is the opponent rung whose score lies closest to 0.500. Anchor ratings come from [Marco Meloni's fixed-node Stockfish 13 benchmark](#), which connects Stockfish through Fruit 2.2.1 to the historical SSDF scale. The resulting values are benchmark Elo, not FIDE ratings or estimates on a current unrestricted-engine list.

Search budget	Headline score and anchor	Benchmark Elo (95% CI)	Increment
Policy only	0.440 vs 1,000 nodes	1,658 [1,608, 1,710]	--
100	0.480 vs 10,000 nodes	2,456 [2,400, 2,512]	+798
1,000	0.450 vs 50,000 nodes	2,925 [2,875, 2,977]	+469
10,000	0.520 vs 100,000 nodes	3,114 [3,065, 3,163]	+189
100,000	0.530 vs 200,000 nodes	3,251 [3,206, 3,297]	+137

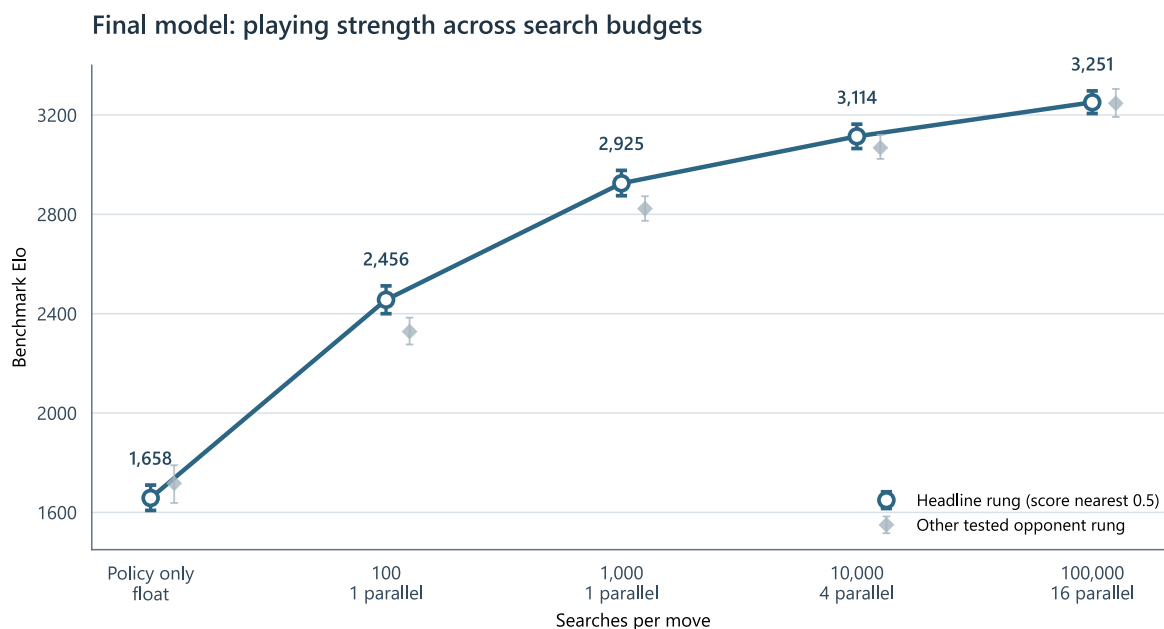


Figure 7.4: The connected points use the opponent rung with score nearest 0.5; pale diamonds show the other measured rung at each budget. Vertical bars are the reported 95% confidence intervals. The categorical horizontal axis keeps policy-only play visible alongside the searched conditions; it does not imply equal compute spacing. Policy-only uses the float export, while searched points use INT8 TensorRT. The parallel-search count also changes with budget, so this is a measured operating curve rather than an isolated node-budget experiment.

Search therefore adds 1,593 benchmark Elo from policy-only play to the deepest measured condition, with diminishing returns at each decade. The 100,000-search estimate is unusually well anchored: an independent 100,000-node opponent gives 3,247 Elo, only four points below the 200,000-node result. That local agreement supports the top headline, but does not validate extrapolation beyond the measured anchors.

The complete ten-row matrix, including W/D/L and archive-capture status, is in the [result record](#).

Two protocol effects that matter

Opponent rungs disagree at lower budgets

At 100, 1,000, 10,000, and 100,000 searches, the harder opponent rung reads 128, 102, 46, and 4 Elo higher than the easier rung. Ideal transitive Elo would give the same estimate from both. Draw behavior, anchor calibration error, matchup effects, and sampling noise are possible contributors, but the project did not isolate the cause. Choosing the score closest to 0.5 limits extrapolation. The disagreement matters at low budgets but is immaterial to the deepest result.

Parallelism buys time by spending strength

The headline curve is an operating curve, not a pure search-budget ablation: it uses one parallel search at 100

and 1,000 searches, four at 10,000, and sixteen at 100,000. A controlled 1,000-search sweep measured 2,823 Elo with one parallel search, 2,804 with four, and 2,778 with sixteen. Four-way parallelism therefore cost 19 Elo; sixteen-way cost 45 Elo.

The two recorded timing totals give **18.1–18.5 minutes** with one parallel search, **3.4–3.8** with four, and **1.3–1.8** with sixteen. These are descriptive ranges across the operational recap and result manifests, not confidence intervals. The corresponding paired speedup ratios span approximately **4.8–5.3x** for four-way and **10.1–13.9x** for sixteen-way parallelism. At only 100 searches, sixteen-way parallelism cost 235 Elo, so parallelism cannot be changed silently across a compute curve. The available points suggest that a fixed parallel count becomes less harmful as the total budget grows, but they are too sparse to define how much parallelism is effectively free at each budget. That frontier would require a dedicated budget-by-parallelism grid.

What the student establishes—and what it does not

Both completed students have **470,295 parameters**, 13.4 times fewer parameters than the teacher. They differ only in training duration on the same 20-million-row replay snapshot. At 10,000 searches against the same 20,000-node anchor, the 36,621-step student scored 31/33/36 for **2,683**

Elo [2,637, 2,731]; the 110,000-step student scored 35/29/36 for **2,697 [2,640, 2,753]**. Tripling training from exactly 7.500 to 22.528 replay epochs therefore moved the point estimate by 14 Elo, well inside the confidence intervals.

The longer student's training/held-out policy losses ended at 1.8613/1.8815, a roughly 0.020 gap that was flat from about step 60,000. Combined with the match, this suggests that the small architecture had saturated on this dataset; it does not demonstrate a memorisation collapse.

At 100,000 searches the longer student scored 59/28/13 against the 20,000-node anchor, corresponding to **2,873 Elo [2,819, 2,935]**. The planned 50,000-node bracket was skipped, so this remains an unbracketed lower anchor-based estimate. The student used TorchScript and the teacher INT8 TensorRT, further preventing a clean architecture-only comparison. Elo is an interval scale: parameter compression can be expressed as 13.4x, but ratings cannot be meaningfully expressed as one model having a percentage of another model's Elo.

8. What the evidence does not establish

The system reached strong play under a limited single-node training budget, but the final recipe combines many decisions. Search, replay, architecture, quantization, and model sizing changed together. A complete run establishes the performance of that assembly; it does not assign a separate Elo gain to every retained ingredient. Several component comparisons used one seed, a short horizon, or frozen replay. Those studies can establish mechanics or rank a local proxy, but they cannot be promoted into long-run online strength claims.

The terminal ratings are tied to paired games against fixed-node Stockfish 13 and to an external node-to-Elo calibration. They are benchmark Elo, not FIDE ratings or predictions against current unrestricted engines. The finite opening suite and 100-game terminal rows leave sampling uncertainty; two opponent rungs also disagree at some budgets. We show both rungs and select the one closest to a 50% score, but do not claim to know why their inferred ratings differ. The policy-only and searched points use different serving artifacts, and the deep-search points use different degrees of parallelism. The plotted curve is a measured operating curve, not an isolated search-budget scaling law.

The economics of rejected methods belong to this workload. Exact graph-state reuse and inference-cache hits were too rare to pay their overhead in diverse chess self-play. Adaptive stopping saved simulations but barely shortened the learning cycle because training overlapped self-play. A repetitive analysis service, another game, or a non-overlapped learner could change those conclusions. Similarly, the matched policy-head and trunk screens do not prove that convolution is universally preferable to attention.

Some historical policy-head and global-context comparisons survive only as owner recollection rather than complete result artifacts. Their qualitative interpretation is separated from preserved measurements in the investigation chapters. The larger-network continuation reached parity but not a clear gain; its limited duration cannot separate an optimization problem from a capacity or

target-quality limit. The final student experiments show useful compression, but did not test a student and teacher under identical deployment backends at every budget.

The selected-checkpoint training-volume and trainer-throughput figures are reconciled, but wider end-to-end self-play/search throughput and total project expenditure are not. The known **\$43.20** is only the accepted 60-hour lineage at **\$0.72/h**, not the total cost of experiments, evaluation, or discarded work. Proprietary drivers and TensorRT versions also limit bitwise reproduction: the intended reproducibility target is a recorded recipe, artifacts, protocol, and statistical agreement, not identical future weights.

9. Reproducibility

Two reproducibility targets

The project maintains two distinct targets:

1. **Recipe reproduction:** use the current fully expanded `chess-final-config.yaml` as the supported entry point.
2. **Result reproduction:** use the frozen source revision, resolved config hash, manifest, checkpoints, engines, datasets, and archive recorded for the final result.

The first may evolve; the second must not.

Result identity and provenance

The frozen result record and its linked evidence identify these layers of provenance:

- Git source revision and clean/dirty state;
- resolved YAML and SHA-256;
- dependency lock and hash;
- operating image, Python, PyTorch, CUDA, cuDNN, driver, GPU, CPU, RAM, and disk facts;
- Stockfish and KataGo versions and archive hashes where applicable;
- evaluation dataset and opening-suite identities and hashes;
- run manifest, approval record, coordinator logs, TensorBoard events, and resource telemetry;
- replay schema, final capacity/occupancy, and reconciled volume counters;
- selected training checkpoint and trimmed inference artifact hashes;
- ONNX, TensorRT template/engine provenance, calibration positions, and fidelity reports;
- raw terminal match records, aggregate reports, commands, and confidence-interval method;
- one digest covering the fetched archive or a checksummed artifact manifest.

The authoritative values belong in the [final result record](#) and Chapter 7.

Reproducing the software

Local setup and validation begin in the root [README](#), the [Python guide](#), and the [native runtime guide](#). Production nodes are provisioned by `deployment/setup_remote.sh`, which installs locked dependencies, builds the Release extension, installs pinned

evaluation engines, and runs engine smokes. Run lifecycle operations go through `deployment/run_control.sh`.

This chapter intentionally does not duplicate commands from current operational documentation. The `experiment platform`, `run control`, and `result export` documents are the executable authorities.

Original project code and documentation, including this report, are available under the repository [MIT License](#). The published final model artifacts carry the same license in the [Hugging Face model repository](#). External dependencies, reference sources, and third-party data retain their own terms.

Reproducing evaluation

Use the preserved checkpoint and inference artifact rather than re-exporting it with a newer toolchain. Reuse the same paired opening suite, colors, opponent binary, node limit, threads, hash, candidate search budget, parallelism, batching, and adjudication rules. Report every game and recompute aggregates independently.

For latency, separate:

- isolated model-forward throughput;
- saturated many-position search throughput;
- single-game interactive latency.

Only compare like with like. The `Stockfish gauntlet` defines the current match procedure, while `evaluation engines` owns binary identity.

Reproducing plots and tables

The plots derive from archived JSON, CSV, TensorBoard, or manifest data. Figure inputs and extraction methods are retained beside the final evidence archive. Derived tables preserve the raw columns needed to recompute totals, rates, Elo transformations, and uncertainty intervals; live-dashboard values are not treated as publication data.

10. Conclusion

An AlphaZero-style chess player trained from random initialization can become very strong on a tightly limited compute budget, provided its entire learning loop is engineered to turn GPU time into useful games, reliable targets, and measurable improvement. The selected 6.3-million-parameter model reached **3,251 benchmark Elo at 100,000 searches per move** in the project's fixed-node Stockfish 13 protocol, with a **1,658 benchmark-Elo policy-only** result. A matched-estimator comparison places its 64-search training plateau about **74 Elo** above the previous baseline. These are protocol-specific measurements, not FIDE ratings or claims against unrestricted engines.

The result was not produced by one isolated trick. Compact structured policy prediction, a shared convolutional network, progressive small-to-medium sizing, searched self-play with restart-state and replay selection, and an INT8-capable architecture all mattered to the assembled recipe. Native tree ownership, batched inference, TensorRT, and distributed training made the volume of searched data feasible. The report does not infer an individual Elo contribution where only a component proxy or the final bundle was measured.

The negative results sharpen that conclusion. A learned allocator produced better deep-policy fidelity but worse online learning. A stopper saved simulations but barely reduced wall-clock cycle time. Exact graph and neural-cache reuse were too sparse for their overhead in this chess workload. More parallel leaves sped service while spending some playing strength, especially at shallow budgets. Three failures exposed the coupling that those local metrics miss: omitting searched endgames poisoned value targets, a mechanically successful TensorRT refit changed model behavior, and incomparable training losses promoted a much weaker candidate.

The strongest general lesson is to measure the whole loop. For every proposed saving, ask how many additional reliable positions it creates, how quickly the learner absorbs them, and whether the deployed artifact actually plays better. The remaining uncertainty is also clear: the larger model recovered its parent's strength without surpassing it in the available continuation, and the compact student preserved substantial strength without matching the teacher. Neither outcome closes the question of how much more chess strength the same framework could gain from longer training, different targets, or more capacity.

References

- Silver, D. et al. (2017). *Mastering Chess and Shogi by Self-Play with a General Reinforcement Learning Algorithm*. [arXiv:1712.01815](#).
- Silver, D. et al. (2017). *Mastering the Game of Go without Human Knowledge*. *Nature* 550, 354–359. [doi:10.1038/nature24270](#).
- Wu, D. J. (2020). *Accelerating Self-Play Learning in Go*. [arXiv:1902.10565v5](#).
- Tian, Y. et al. (2019). *ELF OpenGo: An Analysis and Open Reimplementation of AlphaZero*. [arXiv:1902.04522v5](#).
- Lan, L.-C. et al. (2021). *Learning to Stop: Dynamic Simulation Monte-Carlo Tree Search*. [arXiv:2012.07910](#).
- Czech, J. et al. (2024). *Representation Matters for Mastering Chess: Improved Feature Representation in AlphaZero Outperforms Switching to Transformers*. [arXiv:2304.14918v2](#).
- Wu, T.-R. et al. (2024). *MiniZero: Comparative Analysis of AlphaZero and MuZero on Go, Othello, and Atari Games*. [arXiv:2310.11305v3](#).
- Fedus, W. et al. (2020). *Revisiting Fundamentals of Experience Replay*. [arXiv:2007.06700](#).
- D'Oro, P. et al. (2023). *Sample-Efficient Reinforcement Learning by Breaking the Replay Ratio Barrier*. [ICLR paper](#).
- Schaul, T. et al. (2015). *Prioritized Experience Replay*. [arXiv:1511.05952](#).
- Jones, A. L. (2021). *Scaling Scaling Laws with Board Games*. [arXiv:2104.03113](#).
- Grill, J.-B. et al. (2020). *Monte-Carlo Tree Search as Regularized Policy Optimization*. [arXiv:2007.12509](#).
- Danihelka, I. et al. (2022). *Policy Improvement by Planning with Gumbel*. [ICLR paper](#).
- Trudeau, A. and Bowling, M. (2023). *Targeted Search Control in AlphaZero for Effective Policy Improvement*. [arXiv:2302.12359](#).
- Schrittwieser, J. et al. (2021). *Online and Offline Reinforcement Learning by Planning with a Learned Model*. [arXiv:2104.06294](#).
- Tsai, Y.-J. et al. (2026). *Regret-Guided Search Control for Efficient Learning in AlphaZero*. [arXiv:2602.20809v1](#).
- Czech, J., Korus, P., and Kersting, K. (2020). *Monte-Carlo Graph Search for AlphaZero*. [arXiv:2012.11045](#).
- KataGo project (release 1.17.1). [Methods documentation, self-play training, and training history](#).
- NVIDIA. [TensorRT quantization, engine refitting, and 10.14.1 release notes](#).
- Meloni, M. (2021). *Stockfish and Lc0, test at different number of nodes. Calibration benchmark*.